

My dissertation title spanning two lines

My Full Name

My College

This dissertation is submitted for the degree of Doctor of Philosophy

Abstract

My abstract ...

Declaration

This dissertation is the result of my own work and includes nothing which is the outcome of work done in collaboration except as declared in the Preface and specified in the text. It is not substantially the same as any that I have submitted, or am concurrently submitting, for a degree or diploma or other qualification at the University of Cambridge or any other University or similar institution except as declared in the Preface and specified in the text. I further state that no substantial part of my dissertation has already been submitted, or is being concurrently submitted, for any such degree, diploma or other qualification at the University of Cambridge or any other University or similar institution except as declared in the Preface and specified in the text. This dissertation does not exceed the prescribed limit of 60 000 words.

My Full Name
My Month, My Year

Acknowledgements

My acknowledgements ...

Contents

Chapter 1

Introduction

1.1 Problem Importance

Modern digital environments rely on multiple independent services and applications to perform everyday operations such as monitoring, data synchronization, notifications, and reporting. These operations are often repetitive and require frequent execution, either in response to specific events or according to fixed schedules. Performing such tasks manually consumes significant time, increases operational overhead, and introduces the possibility of human error.

Workflow automation has therefore become an essential approach to improving efficiency and reliability in modern systems. Automation tools enable users to define sequences of tasks that execute automatically, reducing manual intervention and allowing systems to operate continuously. The importance of automation grows as systems become more interconnected and data-driven, making flexible and scalable workflow orchestration a critical requirement in many domains.

1.2 Challenges

Despite the growing adoption of automation platforms, several challenges remain. First, integrating multiple services with different interfaces and data formats introduces complexity in workflow design and execution. Second, supporting both event-driven and periodic execution requires reliable scheduling mechanisms that can operate autonomously over long periods. Third, many existing solutions are either complex to configure or difficult to extend, limiting their suitability for educational or customizable environments.

Another challenge lies in designing a modular architecture that supports future expansion without affecting core system stability. As automation systems evolve, there is increasing demand to incorporate intelligent processing and interaction with external devices, which requires careful architectural planning from the beginning.

1.3 Domain

This project belongs to the domain of workflow automation and orchestration systems. Workflow automation focuses on defining, executing, and monitoring sequences of tasks that connect different services and processes. The project adopts a node-based workflow paradigm in which each node represents a specific operation or integration step.

The domain intersects with distributed systems, event-driven architecture, and task scheduling. Additionally, the system is designed with extensibility in mind, allowing future integration with emerging domains such as artificial intelligence services and Internet of Things (IoT) devices.

1.4 Proposed Solution

The proposed solution is a workflow automation platform inspired by node-based automation tools. The system allows users to create workflows composed of interconnected nodes that execute tasks automatically based on triggers or scheduled intervals.

The platform focuses on providing a clear separation between workflow definition, execution logic, and integration components, ensuring modularity and maintainability. Core features include event handling, periodic task scheduling, workflow execution management, and monitoring capabilities.

While automation and scheduling form the core functionality of the system, the architecture is intentionally designed to support optional extensions such as AI nodes for intelligent processing and IoT nodes for device interaction. These extensions demonstrate the flexibility of the proposed architecture without redefining the core objectives of the project.

1.5 Limitations

The scope of this project is limited to implementing the fundamental concepts of workflow automation and orchestration. The system is intended as an academic and practical prototype rather than a full commercial platform.

Therefore, certain advanced capabilities are out of the scope of this work, including large-scale distributed execution, enterprise-grade security policies, advanced access control mechanisms, and extensive third-party integration marketplaces. AI and IoT components are included only as extensibility demonstrations and are not required for core system operation.

1.6 Target Users

The primary target users of the proposed system are students, developers, and small teams who require a flexible and understandable automation platform for integrating services and executing repetitive tasks. The system is designed to support users who may not wish to build complex automation pipelines manually but still need customization and control over workflow behavior.

By focusing on usability and extensibility, the project aims to provide a learning-friendly automation environment that can serve both educational and practical experimentation purposes.

Chapter 2

Background, Related Work and Comparison

2.1 Background

2.1.1 Workflow Automation

Workflow automation refers to the process of defining and executing sequences of tasks automatically with minimal human intervention. Automation systems are widely used to reduce repetitive manual operations, improve consistency, and increase operational efficiency. Workflows are typically triggered either by specific events or by scheduled intervals, enabling systems to operate continuously and autonomously.

Modern automation platforms allow users to connect multiple services and define logic that determines how data and actions flow between them. This approach improves scalability and reduces the complexity of managing interconnected systems.

2.1.2 Node-Based Workflow Architecture

Many contemporary automation systems adopt a node-based architecture in which each node represents a specific task or integration step. Nodes are connected to form workflows that define execution order and data flow.

The node-based model provides modularity, reusability, and clarity, allowing users to build complex automation pipelines from simple components. This architecture also simplifies extensibility, since new functionality can be introduced through additional nodes without modifying the core system.

2.1.3 Scheduling and Periodic Execution

In addition to event-driven workflows, automation systems often require periodic execution of tasks. Scheduling mechanisms enable workflows to run at predefined intervals, supporting use cases such as data polling, system monitoring, report generation, and synchronization processes.

Reliable scheduling is an essential component of workflow orchestration, as it allows automation systems to function independently over long periods without direct user interaction.

2.1.4 Extensible System Design

Extensibility is a key requirement for modern automation tools. A modular architecture allows the system to evolve over time by adding new integrations and capabilities. Designing workflows around interchangeable nodes enables future support for advanced functionalities such as intelligent processing or device interaction while preserving core stability.

2.2 Related Work

Several automation platforms have influenced the development of workflow orchestration systems. These platforms demonstrate different design philosophies and capabilities.

2.2.1 n8n

n8n is an open-source workflow automation platform that uses a node-based interface to connect various services. It provides flexibility and extensibility through customizable nodes and supports both event-based and scheduled workflows. However, advanced configurations may require technical knowledge.

2.2.2 Zapier

Zapier is a cloud-based automation service focused on simplicity and ease of use. It offers a large number of integrations and a user-friendly interface. Nevertheless, its closed architecture limits customization and extensibility compared to open-source alternatives.

2.2.3 Node-RED

Node-RED is a flow-based programming tool originally designed to connect hardware devices, APIs, and online services. It emphasizes visual programming and is commonly used in IoT scenarios. Although highly flexible, it may require additional configuration for complex workflow orchestration.

2.2.4 Apache Airflow

Apache Airflow is a workflow orchestration platform designed primarily for data engineering and pipeline scheduling. It provides powerful scheduling and monitoring capabilities, but targets technical users and large-scale workflows rather than lightweight automation scenarios.

2.3 Comparison of Existing Solutions

Table ?? presents a comparison between the existing platforms and the proposed system.

Table 2.1: Comparison of Workflow Automation Platforms

Feature	n8n	Zapier	Node-RED	Proposed System
Event-driven workflows	Yes	Yes	Yes	Yes
Periodic scheduling	Yes	Limited	Yes	Yes
Open architecture	Yes	No	Yes	Yes
Ease of learning	Medium	High	Medium	High
Extensible node system	Yes	Limited	Yes	Yes
AI/IoT extensibility	Partial	Limited	Good	Planned

2.4 Research Gap and Novelty

Existing automation platforms provide powerful capabilities to connect services and execute workflows. However, many of these solutions prioritize enterprise-scale features or advanced configurations, which can increase complexity for educational or experimental use.

The analysis of previous work reveals a gap in systems that emphasize clarity of architecture, simplified workflow orchestration, and extensibility for future enhancements while maintaining a lightweight core design.

The novelty of the proposed system lies in presenting a modular automation architecture focused on workflow execution and scheduling as primary components, while intentionally enabling future extension through additional nodes such as artificial intelligence and IoT integrations. This approach balances simplicity and extensibility, making the system suitable for both learning purposes and practical experimentation.

Chapter 3

Implementation

3.1 Trigger Nodes Mechanism

3.1.1 Overview

In a workflow automation platform, every workflow must have an entry point — something that decides when execution begins. This role is fulfilled by trigger nodes. A trigger node is a specialized node that listens for a specific event or condition and, when that condition is met, fires the rest of the workflow graph for execution. Unlike regular action nodes that simply process inputs and pass outputs downstream, trigger nodes are always-on components that run persistently as long as a workflow is deployed. Each trigger type reacts to a different source of events. The platform supports the following trigger types:

- **Cron** — fires repeatedly on a fixed time interval, starting from a configurable date.
- **Poll** — periodically checks an external source for new data.
- **Webhook** — waits for an incoming HTTP request before firing.
- **Manual** — fires immediately on demand when a user triggers execution directly.

The trigger mechanism is built around three key concerns: starting triggers when a workflow is deployed, reliably passing execution down the workflow graph when a trigger fires, and cleanly stopping all triggers when a workflow is stopped or the server shuts down.

3.1.2 Core Abstractions

3.1.2.1 The EventSource Interface

All trigger types implement the `EventSource` interface, which defines a minimal two-method contract:

```

1 // eventSource.ts
2 import { Job } from 'bullmq';
3
4 export interface EventSource {
5   start(body: Record<string, unknown>): Promise<Job>;
6   stop(): Promise<void>;
7 }

```

This interface was intentionally kept minimal. The `start()` method receives the node configuration and returns a BullMQ Job handle, while `stop()` is responsible for cleanly cancelling whatever mechanism the trigger started. Any new trigger type only needs to implement these two methods, making the system open to extension without modifying existing code.

Enforcing the same `start/stop` contract across all trigger types means the workflow runner can manage cron and poll triggers through a single loop, without any conditional branching based on type. Adding a new trigger type in the future requires no changes to the runner — only a new class implementing `EventSource`.

3.1.2.2 The Node Base Class

Trigger nodes are still Node subclasses, following the same abstract base class as all other node types in the system. What distinguishes a trigger node is the `triggerNode: true` flag in its static description object, along with a type of 'cron', 'poll', etc.

```

1 // nodeBase.ts (excerpt)
2 export interface INodeDescription {
3   name: string;
4   displayName: string;
5   type: 'cron' | 'curl' | 'webhook' | 'generic' | 'poll';
6   category: 'ai' | 'action-in-app' | 'data-transformation' | '
       core' | 'flow-control';
7   inputs: string[];
8   outputs: string[];
9   credentials?: string[];
10  parameters?: any[];
11  triggerNode?: boolean; // marks this node as a trigger
12 }
13
14 export abstract class Node {
15   static description: INodeDescription;
16   abstract execute(inputs: any): Promise<any>;

```

```

17 poll?(): Promise<any>;
18 webhook?(request: any): Promise<any>;
19 }

```

The `execute()` method is abstract and must be implemented by every node, including trigger nodes. For trigger nodes, this method represents what happens when the trigger actually fires — for a cron node it simply returns a confirmation string, since its real work is handled by the scheduler.

3.2 The Cron Trigger

The `CronSource` class implements `EventSource` for time-based, repeating workflow execution. It wraps BullMQ's job scheduler API to create persistent, interval-based jobs that survive application restarts.

3.2.1 Configuration Parameters

The cron node accepts two parameters defined in its static description: The `start()`

Table 3.1: Schedule configuration parameters

Parameter	Description
<code>interval</code>	Repeat interval in milliseconds (e.g., 60000 for 1 minute)
<code>start</code>	ISO timestamp or epoch ms — the reference point for the schedule

method extracts these from the node's parameters array and validates them before scheduling anything:

```

1  const intervalParam = this.node.parameters?.find(p => p.title ===
   'interval');
2  const startParam    = this.node.parameters?.find(p => p.title ===
   'start');
3
4  const interval = intervalParam.value ?? intervalParam.default;
5  const start    = startParam.value    ?? startParam.default;
6
7  if (typeof interval !== 'number' || interval <= 0)
8    throw new AppError('Invalid trigger parameters', 403);
9
10 const startDate = new Date(start).getTime();
11 if (isNaN(startDate))
12   throw new AppError('Invalid start date', 403);

```

3.2.2 Delay Calculation

A cron trigger does not simply fire at interval ms from now. Instead, it respects the configured start date as an anchor point. If the start date is in the past, the system calculates when the next execution should have occurred and schedules from there:

```
1 const now = Date.now();
2
3 let delay;
4 if (now >= startDate) {
5   // How many full intervals have elapsed since start?
6   const intervalsPassed = Math.floor((now - startDate) / interval
7     );
8
9   // Next execution is at the boundary after current time
10  const nextExecTime = startDate + ((intervalsPassed + 1) *
11    interval);
12
13  delay = Math.max(0, nextExecTime - now);
14 } else {
15   // Start date is in the future - just wait for it
16   delay = startDate - now;
17 }
```

This ensures that a cron trigger which was configured to run every 10 minutes starting at 09:00 will, if deployed at 09:47, fire next at 09:50 — not 09:57. The trigger stays aligned with the intended schedule regardless of when the workflow was deployed.

3.2.3 Scheduling with BullMQ

BullMQ's `upsertJobScheduler()` is used to register the repeating job:

```
1 const job = await this.queue.upsertJobScheduler(
2   this.jobId,
3   {
4     every: interval, // repeat every N milliseconds
5     startDate: startDate,
6   },
7   {
8     name: 'workflow-job',
9     data: {
10       workflowId: this.node.projectId,
11       nodeId: this.node.id,
```



```

12     nodeName:    this.node.name,
13     nodeType:    this.node.type,
14     children:    this.children,
15     timestamp:   Date.now()
16   }
17 }
18 );

```

The use of `upsertJobScheduler` rather than `add()` is deliberate: it is idempotent. If a scheduler with the same `jobId` already exists (e.g. after a server restart or a re-deploy), it is updated in place rather than creating a duplicate. The `jobId` follows the convention `cron-projectId-nodeId`, which makes it unique per node per project.

3.2.4 Stopping the Cron Trigger

Stopping the cron trigger is straightforward — the scheduler is removed by its ID:

```

1 async stop() {
2   if (!this.jobId)
3     throw new AppError('No scheduler ID available', 500);
4
5   const removed = await this.queue.removeJobScheduler(this.jobId)
6     ;
7
8   if (removed) console.log('Stopped cron scheduler: ${this.jobId}');
9   else        console.log('No scheduler found: ${this.jobId}');
10 }

```

This permanently removes the repeating schedule from BullMQ's Redis store. Any jobs already in the queue from this scheduler may still execute, but no new ones will be added.

3.2.5 BullMQ over native timers

Using `setInterval()` or `setTimeout()` for cron scheduling would not survive a server restart — all in-flight timers are lost when the process exits. BullMQ persists job schedules in Redis, meaning that if the server crashes and restarts, scheduled jobs resume automatically without any extra recovery logic.

3.3 The Poll Trigger

The Poll Trigger is designed to monitor external data sources by periodically checking for updates. Unlike event-driven triggers such as webhooks, which rely on external systems to push events, the poll trigger actively requests data at fixed intervals. This makes it particularly useful when integrating with systems that do not support real-time notifications.

The trigger operates by executing a polling loop that runs at a configurable interval. At each iteration, it queries the target resource and determines whether new data is available. If a change is detected, the trigger fires and initiates a new workflow execution.

3.3.1 Poll Node Structure

The Poll Trigger is implemented as a specialized node class named `PollNode`, which extends the base `Node` abstraction. It is marked as a trigger node using the `triggerNode: true` flag within its static description, allowing the workflow engine to treat it as an entry point.

The node metadata is defined using the `INodeDescription` interface, ensuring consistency with other node types in the system.

3.3.2 Configuration Parameters

The Poll Node exposes several parameters that allow users to configure how the polling process behaves. These parameters are defined declaratively within the node description and are rendered dynamically in the user interface.

- **Service:** Specifies the external service to be monitored (e.g., Gmail, Github ...).
- **Email:** The account used to access the service.
- **App Password:** Authentication credential required for secure access.
- **Interval:** Defines the polling frequency in milliseconds (default: 300000 ms, i.e., 5 minutes).
- **From Email (Optional):** Filters incoming data based on sender address.
- **Subject Contains (Optional):** Filters messages based on subject keywords.

This design allows flexible filtering of incoming data, reducing unnecessary workflow executions.

3.3.3 Separation of Responsibilities

Although the `PollNode` defines the configuration and identity of the trigger, it does not implement the actual polling logic. Instead, the execution responsibility is delegated to a separate component within the trigger system (e.g., `PollSource`), which implements the `EventSource` interface.

This separation follows a clean architectural pattern:

- **PollNode**: Defines configuration and integrates with the workflow graph.
- **PollSource**: Handles polling logic, scheduling, and event detection.

3.3.4 Polling Logic and Execution Flow

The actual polling mechanism is implemented in the trigger system (via `PollSource`). When the workflow is deployed:

1. The workflow engine detects that the node is a trigger node of type `poll`.
2. It initializes the corresponding `PollSource`.
3. The `start()` method is invoked, which:
 - Creates a periodic loop using the configured interval.
 - Connects to the specified external service.
 - Retrieves new data (e.g., emails).
 - Applies filtering conditions (`fromEmail`, `subjectContains`).
4. If new matching data is detected:
 - A new job is enqueued using BullMQ.
 - The workflow execution begins from this trigger node.

When the workflow is stopped, the `stop()` method is called to terminate the polling loop and release any associated resources.

3.3.5 Extensibility Considerations

The Poll Node is designed to be easily extended. Additional services (e.g., GitHub, Slack) can be supported by enhancing the polling logic in `PollSource`, without requiring changes to the node structure itself.

Future improvements may include service-specific validation rules, such as validating token formats depending on the selected service.

3.3.6 Design Evaluation

This implementation demonstrates a clear separation between configuration, execution, and event handling. Such a design provides several advantages:

- Improved maintainability through separation of concerns.
- Independent scaling of trigger execution.
- Ease of adding new trigger types or services.
- Lightweight and declarative node definitions.

3.4 Backend Architecture

3.4.1 Backend Overview

The backend of the FLOO platform is built on a modular, multi-layered architecture using Node.js, Express.js, and TypeScript. This design enforces a strict separation of concerns, ensuring that routing, security, business logic, and execution are decoupled. When an HTTP request enters the system, it traverses a well-defined pipeline of middlewares and validators before reaching the core business logic.

3.4.2 Request Lifecycle and Flow

The lifecycle of a typical API request follows a strict sequential path designed to fail fast on invalid or unauthorized operations. The flow ensures that the core services and execution engine only ever process sanitized, authorized, and well-formed data.

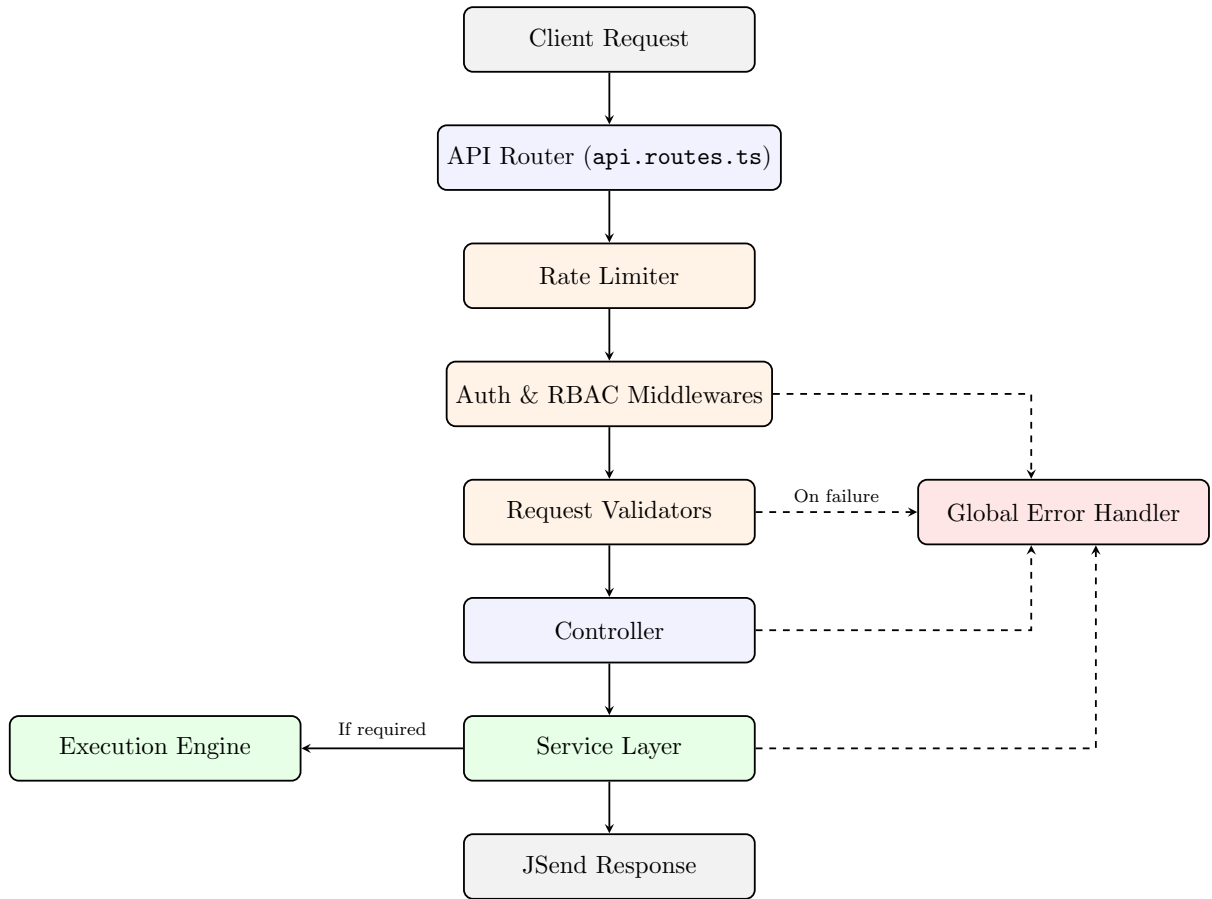


Figure 3.1: High-level lifecycle of an incoming API request

3.4.3 Backend System Layers

The directory structure directly reflects the architectural boundaries of the application. Each layer has a single, distinct responsibility.

3.4.3.1 Routes

The routing layer is responsible for mapping incoming HTTP endpoints to their corresponding controller handlers. It acts as the entry point for the API hierarchy. Routes are logically grouped by entity (e.g., users, projects, workflows, nodes) and define the specific middleware chain required for each endpoint.

By keeping the routing layer declarative, it provides a clear, high-level map of the entire API surface without bleeding into implementation details.

3.4.3.2 Middlewares

Middlewares act as the gatekeepers of the application. They intercept incoming requests to enforce systemic rules before the request can reach the business logic. This layer handles security concerns such as JSON Web Token (JWT) verification, role-based access control (RBAC), and email verification checks.

Additionally, middlewares manage traffic control via strict rate limiters, standardize outgoing success responses using the JSend format, and intercept exceptions through a centralized, registry-based error handling mechanism.

3.4.3.3 Validators

Request validation is abstracted into its own layer using `express-validator`. Before a request payload is processed, it is checked against strict schemas to ensure data integrity and type safety.

This layer ensures that bad payloads, missing parameters, or malformed data are rejected immediately with a `422 Unprocessable Entity` response. This "fail-fast" approach guarantees that the underlying controllers and services do not need to perform redundant sanity checks on incoming data.

3.4.3.4 Controllers

Controllers serve strictly as orchestrators between the HTTP transport layer and the business logic. A controller's only responsibility is to extract data from the incoming request (headers, body, params, query), pass that data to the appropriate service, and return the service's output back to the client.

Controllers are intentionally kept thin; they do not contain complex business rules or database queries, adhering strictly to the Single Responsibility Principle.

3.4.3.5 Services

The service layer forms the heart of the backend, encapsulating the core business logic of the FLOO platform. All heavy lifting—such as workflow graph construction, file processing with Cloudinary, communicating with the LLM via Groq, and interacting with the database—happens here.

By isolating business logic into services, the code becomes highly reusable and independently testable. For example, the execution engine can invoke a service method directly without needing to mock an HTTP request.

3.4.3.6 Strategies

The strategies layer encapsulates interchangeable authentication algorithms and behaviors, specifically leveraging Passport.js for OAuth 2.0 integrations.

This directory contains the logic required to negotiate with external identity providers (such as Google, GitHub, and Discord) and third-party APIs like Google Sheets. Separating these into strategies allows the platform to easily scale the number of supported integrations without cluttering the core authentication flow.

3.4.3.7 Utils

The utils directory houses pure, stateless helper functions that are shared across multiple layers of the application. These include cryptographic operations like hashing passwords with `bcryptjs`, generating JWTs, and creating one-time passwords (OTPs) for email verification.

Because these functions are pure and self-contained, they are highly reliable and prevent code duplication throughout the codebase.

3.4.3.8 Types

TypeScript interfaces and type definitions form the foundational contract of the application. The types layer defines the exact shape of data structures used across the system, such as augmented Express Request objects (`AuthRequest`) or standardized OAuth profiles.

This layer ensures strict compile-time safety, allowing the compiler to catch structural mismatches between the frontend payloads, the internal API representation, and the execution engine.

3.5 Data Layer Architecture

3.5.1 Database Requirements

The FLOO platform requires a robust, relational data storage layer capable of supporting multi-tenant isolation, complex configuration storage, and strict security compliance. The primary storage needs encompass:

- **User and Identity Management:** Tracking local user profiles, multi-provider OAuth identities, and RBAC roles to ensure secure, isolated workspaces.
- **Project and Workflow Configurations:** Storing dynamic, graph-based automation logic and tracking deployment states.

- **Execution Analytics:** Persisting detailed logs of workflow runs, statuses, and generated outputs (such as Cloudinary asset links) for debugging and system analytics.
- **Secure Credential Management:** Safely storing third-party integration secrets (e.g., API keys, OAuth tokens) utilizing an encrypted, dynamically generated structure.

Key Drivers: Relational integrity is paramount due to the highly interconnected nature of the platform. For example, a User owns Projects, which trigger Executions, which in turn generate NodeOutputs. Transactional safety is explicitly required for operations like cascading account deletions, ensuring that scrubbing a user removes all associated relational assets without creating orphaned records. Prisma was selected as the ORM to enforce these strict foreign key constraints while providing end-to-end type safety with the TypeScript backend.

3.5.2 Conceptual Design (ERD)

The Conceptual Entity Relationship Diagram (CERD) outlines the high-level business entities and their interactions.

3.5.2.1 Core Entity Relationships

The conceptual model is heavily centered around the **User** entity, enforcing a strict ownership hierarchy:

- **Users to Projects (1:N):** A single user can create and manage multiple automation projects. All workflow executions, chat histories, and node interactions branch off this fundamental relationship.
- **Projects to Executions (1:N):** Every time a workflow is triggered, an execution record is generated to track its lifecycle (e.g., NEW, RUNNING, SUCCESS, ERROR).
- **Nodes to Files (1:N):** External files are represented conceptually as inputs and outputs bound to specific execution instances and nodes, tracking cloud storage references.

3.5.2.2 The Dynamic Credential Architecture

A major architectural redesign was implemented in the credentials subsystem to provide a deep, highly flexible solution that seamlessly bridges backend storage with dynamic frontend UI rendering.

Instead of hardcoding specific integration tokens into a single table, the system abstracts credentials into a definition-and-value pattern:

- **Definition Phase:** Admins define a `CredentialType` (e.g., "Google Sheets" or "SMTP"). This type has a 1:N relationship with `CredentialParameterType`, which outlines the specific fields required (e.g., "API Key", "Client Secret").
- **UI Control Integration:** The `CredentialParameterType` includes metadata fields like `maxChar` and `isSensitive`. This design allows the frontend UI to automatically generate forms with appropriate validation and hidden password inputs without hardcoding frontend logic for every new integration type.
- **Value Phase:** When a user connects an integration, the system generates a `UserCredentialSet` linking to the `CredentialType`. The actual encrypted secrets are stored in `UserCredentialValue` records, creating a normalized mapping back to the expected parameters.

3.5.3 Mapping to Relational Model (Logical/Physical Design)

The logical design translates the conceptual entities into concrete database tables, explicitly defining keys, data types, and constraints utilizing the Prisma schema.

3.5.3.1 Primary and Foreign Key Strategy

- **Universal UUIDs:** All primary keys (PK) across the platform utilize auto-generated UUIDs (e.g., `@id @default(uuid())`). This strategy prevents resource enumeration attacks and ensures global uniqueness, particularly useful when merging distributed data.
- **Composite Constraints:** Certain tables utilize composite keys to enforce logical uniqueness. For example, `AuthIdentityProvider` uses `@@id([provider, providerId])` to ensure a user cannot link the same third-party account twice. Similarly, `UserCredentialValue` enforces `@@unique([setId, paramId])` to guarantee only one value exists per parameter in a given credential set.
- **Aggressive Cascading Deletions:** To maintain strict data hygiene and comply with PII scrubbing requirements, foreign keys aggressively employ `onDelete: Cascade`. Deleting a User cascades down to erase their Projects, which in turn wipes Executions, Node Inputs, and Chat Messages instantly at the database level.

3.5.3.2 Junction and Mapping Tables

The database features specific mapping tables to resolve complex relationships. `UserCredentialValue` serves as the critical junction between a user's `UserCredentialSet` and the system's

`CredentialParameterType`. This normalized approach ensures that if a parameter requirement changes at the system level, the relationship to the user's stored values remains structurally intact.

3.5.3.3 Intentional Denormalization (JSON Workflows)

While the database is heavily normalized, a deliberate exception was made for workflow graph configurations. Rather than splitting workflow nodes, edges, and positions into dozens of relational tables, the entire graph is serialized and stored in a `Json` column within the `Project` table (`workflow Json?`).

This is a strategic design choice: workflow graphs are exclusively read and updated as a single atomic unit by the React Flow frontend. Normalizing the graph would require complex, expensive recursive SQL joins upon every canvas load. Utilizing a JSON column provides the necessary schema flexibility for varied node data while preserving high-performance read/write operations.

3.5.4 Physical Database Design & Implementation

The physical database layer is implemented using PostgreSQL hosted on Supabase. We utilize Prisma ORM to enforce end-to-end type safety across the application. The schema is maintained through declarative migrations, ensuring that the database state is always synchronized with our TypeScript interfaces. Sensitive data, such as integration credentials, is stored using standard text fields, with robust encryption and decryption logic handled strictly at the Service Layer before ever reaching the persistence layer.

3.5.4.1 Database Engine: PostgreSQL (via Supabase)

PostgreSQL was selected as the underlying engine for its strict adherence to ACID compliance, robust support for relational integrity, and advanced JSON processing capabilities. Hosting via Supabase provides managed scaling, automated backups, and connection pooling, ensuring the database can handle high-frequency reads and writes typical of an automated workflow engine.

3.5.4.2 ORM Layer (Prisma)

Prisma acts as the bridge between the Node.js backend and the PostgreSQL database. Rather than writing raw SQL queries, the application relies on the generated Prisma Client.

- **Type Safety:** The `'schema.prisma'` file serves as the single source of truth. Changes to the schema automatically regenerate TypeScript definitions, ensuring that any

mismatched queries, missing fields, or incorrect data types are caught at compile-time rather than throwing unexpected runtime errors.

- **JSON Workflows:** Prisma natively supports PostgreSQL’s ‘JSONB’ data type, allowing the ‘Project’ table to store complex, nested React Flow graphs in the ‘workflow’ column without sacrificing query performance or requiring excessive normalization.

3.5.4.3 Migration Management

Database schema evolution is managed via Prisma Migrations. Instead of executing manual SQL scripts directly against production, developers modify the declarative Prisma schema. The migration engine then generates deterministic SQL differential scripts. This approach ensures that schema changes are strictly version-controlled, reproducible across local, staging, and production environments, and seamlessly integrated into the CI/CD pipeline.

3.5.4.4 Technical Constraints

To guarantee data integrity and query performance at scale, several constraints are baked directly into the physical schema:

Indexes Indexes are strategically applied to fields that are frequently used in ‘WHERE’ clauses, sorting operations, or foreign key lookups to prevent full table scans:

- **Foreign Key Lookups:** Indexes on ‘userId’ within the ‘Project’ and ‘NodeInput’ tables drastically reduce query times when an authenticated user requests their dashboard.
- **Lookup Optimization:** The ‘email’ column in the ‘User’ table is indexed to optimize the high-frequency login and registration authentication flows.
- **Composite Indexes:** High-traffic logging tables utilize composite indexes, such as ‘@@index([executionId, nodeId])’ on ‘NodeOutput’, enabling rapid retrieval of execution assets for the frontend analytics UI.

Constraints

- **Primary Keys:** All entities utilize UUIDs (‘@id @default(uuid())’) as primary keys. This prevents resource enumeration attacks (e.g., guessing a user’s ID) and facilitates distributed data merging.

- **Unique Constraints:** ‘@@unique‘ constraints enforce strict data uniqueness, such as ensuring no two users share an ‘email‘, and no two files share a Cloudinary ‘publicId‘.
- **Composite Uniques:** The ‘UserCredentialValue‘ table employs ‘@@unique([setId, paramId])‘ to guarantee that a specific credential set contains exactly one value per required parameter, preventing duplicate secret injection.

Cascade Rules & Account Deletion Foreign key cascading is aggressively utilized to enforce data hygiene and simplify backend logic. In the schema, relations defining ownership are marked with ‘onDelete: Cascade‘.

This is highly crucial for the platform’s **Account Deletion Flow**. When a user self-deletes (or is purged by an admin), the database engine natively and atomically destroys all linked ‘Project‘ rows. The cascade continues downward, automatically erasing all ‘Execution‘ logs, ‘NodeInput‘ assets, and ‘UserCredentialSet‘ secrets. This guarantees compliance with PII scrubbing requirements without requiring the application service layer to manually fetch and delete thousands of related rows, effectively preventing orphaned data.

3.5.5 Data Dictionary

The following tables detail the physical layout, constraints, and definitions of the core entities in the FLOO system.

3.5.5.1 User Table

Serves as the root entity for identity, access control, and ownership.

Column Name	Type	Constraints	Description
id	UUID	PK	Unique identifier for the user.
email	VARCHAR	Unique, Index	The user's primary email address for authentication.
password	VARCHAR	Nullable	Bcrypt-hashed local password. Null if using strictly OAuth.
isVerified	BOOLEAN	Default: false	Gates access to execution features until email is confirmed.
role	ENUM	Default: USER	Defines RBAC privileges (USER or ADMIN).

Table 3.2: Data Dictionary: User Table

3.5.5.2 Project Table

Stores the configuration and state of the user's workflow automations.

Column Name	Type	Constraints	Description
id	UUID	PK	Unique identifier for the project.
userId	UUID	FK, Cascade	References <code>User.id</code> . Defines ownership.
name	VARCHAR	Not Null	Human-readable name of the automation.
workflow	JSONB	Nullable	Serialized React Flow graph containing node and edge definitions.
deployed	BOOLEAN	Default: false	Tracks whether the project is actively running trigger nodes.

Table 3.3: Data Dictionary: Project Table

3.5.5.3 UserCredentialSet Table

Acts as the parent entity for a group of encrypted third-party integration secrets.

Column Name	Type	Constraints	Description
id	UUID	PK	Unique identifier for the credential set.
userId	UUID	FK, Cascade	References <code>User.id</code> . Determines who can use these credentials.
typeId	UUID	FK, Cascade	References <code>CredentialType.id</code> (e.g., Google Sheets, SMTP).
label	VARCHAR	Not Null	User-defined alias for the credentials (e.g., "My Work Gmail").

Table 3.4: Data Dictionary: UserCredentialSet Table

3.5.5.4 Execution Table

Persists the lifecycle state and telemetry data for individual workflow runs.

Column Name	Type	Constraints	Description
id	UUID	PK	Unique tracking identifier for the job.
projectId	UUID	FK, Cascade	References <code>Project.id</code> . The workflow that was triggered.
status	ENUM	Default: NEW	Tracks the current state (NEW, RUNNING, SUCCESS, ERROR).
error	TEXT	Nullable	Stores execution stack traces or API failure messages if the run crashed.
startedAt	TIMESTAMP	Nullable	The exact time the execution engine picked up the job.

Table 3.5: Data Dictionary: Execution Table

3.6 API & Application Layer

3.6.1 API Overview

The FLOO platform is built on a modular, multi-layered architecture using Node.js, Express.js, and TypeScript, designed to ensure a strict separation of concerns between

routing, business logic, security, and execution.

3.6.1.1 RESTful Conventions

The API is designed around RESTful principles, where resources are mapped to intuitive endpoints categorized by entity (e.g., `/user`, `/project`, `/node`, `/credential`). By maintaining a declarative routing structure, the API surface remains clean and self-documenting, preventing business logic from bleeding into the transport layer. The platform utilizes standard HTTP verbs to perform operations:

- **GET**: Retrieve data or list resources.
- **POST**: Create new resources or trigger actions.
- **PATCH**: Update existing resource metadata.
- **DELETE**: Remove specific resources or clear session data.

3.6.1.2 Authentication and Security

Security is enforced through a tiered middleware system that acts as the application's gatekeeper.

- **JWT Authentication**: Most endpoints require a valid JSON Web Token (JWT) processed by the `auth` middleware to identify the requesting user.
- **Role-Based Access Control (RBAC)**: Sensitive routes utilize the `restrictTo` middleware to enforce fine-grained access based on user roles (`USER` vs `ADMIN`).
- **Verification Gate**: Critical operations, such as managing projects or executing workflows, are protected by the `restrictToVerified` middleware, ensuring only accounts with confirmed emails can perform these actions.
- **Public Access**: Certain endpoints, such as initial Auth/OAuth routes and Webhook triggers, allow public access. These are not left unprotected but are instead secured via dedicated, high-threshold rate limiters.

3.6.1.3 Response Format and Standardization

To ensure consistency across the platform, all responses adhere to the **JSend** format, providing a predictable structure for the frontend.

HTTP Status Codes The API utilizes standard HTTP status codes to communicate outcome states:

Code	Meaning
200/201	Successful request or creation.
400	Bad Request (Missing fields, validation failure).
401	Authentication failed (Expired or invalid token).
403	Forbidden (Insufficient permissions or unverified email).
404	Resource not found.
429	Too Many Requests (Rate limit exceeded).
500	Internal Server Error (Unexpected system failure).

Rate Limiting Traffic control is central to platform stability. The system employs multiple tiers of rate limiting:

- **Standard Limit:** Most API routes are subject to a global limit of 150 requests per 15 minutes to prevent abuse.
- **Action-Specific Limits:** Critical actions have stricter, dedicated limits—such as manual workflow execution (30/min), file uploads (20/15min), or OAuth requests (10/15min)—to protect high-cost resources.
- **Public Limits:** High-volume triggers, like webhooks, utilize a loose limit (e.g., 1000/min) to accommodate frequent event spikes from external providers.

3.6.2 Endpoints Reference

3.6.2.1 Auth Endpoints

This section documents all API endpoints related to user authentication, registration, and session management in FLOO. Most of these endpoints are public but are protected by strict rate limiters and payload validators (`validateRequest`) to ensure security and prevent abuse.

Register User

- **Endpoint:** POST `/api/auth/register`

- **Access:** PUBLIC
- **Description:** Creates a new user account and sends a verification code (OTP) to the provided email. Rate limited to 3 requests per hour per IP.

Field	Location	Type	Description
email	Body	string	Required. User's email address
password	Body	string	Required. User's password
firstName	Body	string	Required. User's first name
lastName	Body	string	Required. User's last name

Table 3.6: POST /api/auth/register — Request Body

Status Code	Condition	Description
201 Created	Success	Verification code sent, returns user and tokens
409 Conflict	Email in use	A user with this email already exists
422 Unprocessable Entity	Validation failed	Required fields missing or invalid
429 Too Many Requests	Rate limit exceeded	Too many accounts created from this IP, please try again after an hour
500 Internal Server Error	Server error	Unexpected failure or email delivery issue

Table 3.7: POST /api/auth/register — Response Codes

Login User

- **Endpoint:** POST /api/auth/login
- **Access:** PUBLIC
- **Description:** Authenticates a user using email and password. Cannot be used for accounts created exclusively via OAuth. Rate limited to 5 attempts per 15 minutes per IP.

Field	Location	Type	Description
email	Body	string	Required. The user's email
password	Body	string	Required. The user's password

Table 3.8: POST /api/auth/login — Request Body

Status Code	Condition	Description
200 OK	Success	Returns user data, access token, and refresh token
401 Unauthorized	Invalid credentials	Wrong email, password, or OAuth-only account
422 Unprocessable Entity	Validation failed	Required fields missing or invalid
429 Too Many Requests	Rate limit exceeded	Too many login attempts from this IP, please try again after 15 minutes
500 Internal Server Error	Server error	Unexpected failure

Table 3.9: POST /api/auth/login — Response Codes

Forgot Password

- **Endpoint:** POST /api/auth/forgot-password
- **Access:** PUBLIC
- **Description:** Generates a 15-minute OTP and sends it to the user's email to reset their password. Always returns a success message for security reasons, even if the email is unregistered. Rate limited to 3 requests per hour.

Field	Location	Type	Description
email	Body	string	Required. The account email address

Table 3.10: POST /api/auth/forgot-password — Request Body

Status Code	Condition	Description
200 OK	Success	Reset code sent successfully
422 Unprocessable Entity	Validation failed	Invalid email format
429 Too Many Requests	Rate limit exceeded	Too many password reset requests, please try again after an hour
500 Internal Server Error	Server error	Unexpected failure or email delivery issue

Table 3.11: POST /api/auth/forgot-password — Response Codes

Verify Reset Code (Reset Password)

- **Endpoint:** POST /api/auth/reset-password
- **Access:** PUBLIC
- **Description:** Validates the OTP sent via email and updates the user's password. The new password cannot match the old password. Rate limited to 3 requests per hour.

Field	Location	Type	Description
email	Body	string	Required. The account email
resetCode	Body	string	Required. The OTP sent to the user
password	Body	string	Required. The new password

Table 3.12: POST /api/auth/reset-password — Request Body

Status Code	Condition	Description
200 OK	Success	Password reset successfully
400 Bad Request	Invalid code	The OTP is incorrect or expired
401 Unauthorized	Same password	The new password is the same as the old one
422 Unprocessable Entity	Validation failed	Missing fields or invalid format
429 Too Many Requests	Rate limit exceeded	Too many password reset requests, please try again after an hour
500 Internal Server Error	Server error	Unexpected failure

Table 3.13: POST /api/auth/reset-password — Response Codes

Refresh Token

- **Endpoint:** POST /api/auth/refresh-token
- **Access:** PUBLIC
- **Description:** Exchanges a valid refresh token for a new access token. Validates against the hashed token stored in the database. Rate limited to 5 requests per 15 minutes.

Field	Location	Type	Description
refreshToken	Body	string	Required. The user's active refresh token

Table 3.14: POST /api/auth/refresh-token — Request Body

Status Code	Condition	Description
200 OK	Success	Returns a new access token
401 Unauthorized	Invalid token	Token is expired, missing in DB, or invalid
429 Too Many Requests	Rate limit exceeded	Too many token refresh requests from this IP
500 Internal Server Error	Server error	Unexpected failure

Table 3.15: POST /api/auth/refresh-token — Response Codes

OAuth Initiation

- **Endpoints:** GET /api/auth/discord, GET /api/auth/google, GET /api/auth/github
- **Access:** PUBLIC
- **Description:** Initiates the OAuth flow by redirecting the user to the respective provider's authentication page. Supports dynamic redirect configuration based on the client type. Rate limited to 10 requests per 15 minutes.

Parameter	Location	Type	Description
client	Query	string	Optional. Pass flutter for mobile deep linking

Table 3.16: GET /api/auth/{provider} — Query Parameters

OAuth Callback

- **Endpoints:** GET /api/auth/discord/callback, GET /api/auth/google/callback, GET /api/auth/github/callback
- **Access:** PUBLIC
- **Description:** Handled by Passport.js strategies. Upon successful authentication, it generates a secure 15-second ticket stored in Redis and redirects to the client application (Web or Flutter) with the ticket appended as a query parameter. Rate limited to 10 requests per 15 minutes.

Status Code	Condition	Description
302 Found	Success	Redirects to client URL with <code>?ticket=UUID</code>
401 Unauthorized	Auth failed	Failed to login with the OAuth provider
429 Too Many Requests	Rate limit exceeded	Too many OAuth requests
500 Internal Server Error	Server error	Unexpected failure

Table 3.17: GET `/api/auth/{provider}/callback` — Response Codes

Exchange Ticket

- **Endpoint:** POST `/api/auth/exchange`
- **Access:** PUBLIC
- **Description:** Exchanges the temporary Redis ticket obtained from the OAuth callback for the actual user data, access token, and refresh token. The ticket is single-use and expires in 15 seconds. Rate limited to 10 requests per 15 minutes.

Field	Location	Type	Description
ticket	Body	string	Required. The UUID ticket from the OAuth redirect

Table 3.18: POST `/api/auth/exchange` — Request Body

Status Code	Condition	Description
200 OK	Success	Ticket exchanged, returns user and tokens
400 Bad Request	Invalid ticket	The ticket is invalid, expired, or already used
422 Unprocessable Entity	Validation failed	Missing ticket in request body
429 Too Many Requests	Rate limit exceeded	Too many OAuth requests
500 Internal Server Error	Server error	Unexpected failure

Table 3.19: POST `/api/auth/exchange` — Response Codes

3.6.2.2 User Endpoints

This section documents all API endpoints related to user management, authentication profiles, and account controls in FLOO. Users and profiles are fundamental entities in the system. All endpoints in this section are protected by the **auth** middleware, requiring a valid JWT token. Role-based access control is strictly enforced via the **restrictTo** middleware.

Note on Verification and Rate Limiting: Unlike project or execution routes, user routes can be accessed by authenticated accounts that are **not yet verified**, allowing them to manage their profile and complete the verification process. All user endpoints are subject to the standard API rate limit of 150 requests per 15 minutes, with stricter dedicated limiters applied to sensitive actions like email verification and password updates.

Get All Users (Admin)

- **Endpoint:** GET /api/user/all
- **Access:** ADMIN only
- **Description:** Retrieves a paginated list of all users stored in the database. Supports dynamic filtering based on account status, roles, and partial text matches for personal details. Subject to the standard API rate limit.

Parameter	Location	Type	Description
disabled	Query	boolean	Exact match filter for disabled status (true/false)
isVerified	Query	boolean	Exact match filter for verification status (true/false)
role	Query	string	Exact match for user role (e.g., ADMIN, USER)
email	Query	string	Case-insensitive partial match for email
firstName	Query	string	Case-insensitive partial match for first name
lastName	Query	string	Case-insensitive partial match for last name
limit	Query	integer	Number of results to return (default: 100)
skip	Query	integer	Number of results to skip for pagination (default: 0)

Table 3.20: GET /api/user/all — Query Parameters

Status Code	Condition	Description
200 OK	Success	Returns a paginated list of users
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Non-admin user	Insufficient permissions
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.21: GET /api/user/all — Response Codes

Get User Profile

- **Endpoint:** GET /api/user/profile
- **Access:** USER, ADMIN
- **Description:** Retrieves the authenticated user’s own profile data. The user ID is automatically extracted from the JWT token. The profile must not be disabled. Subject to the standard API rate limit.

Status Code	Condition	Description
200 OK	Success	Returns the user’s profile
401 Unauthorized	No/invalid token	Authentication failed
404 Not Found	Profile not found	The user profile does not exist or is disabled
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.22: GET /api/user/profile — Response Codes

Get User by ID

- **Endpoint:** GET /api/user/:id
- **Access:** ADMIN, USER
- **Description:** Retrieves a specific user by their ID. Admins receive full details (including disabled accounts and sensitive fields), while standard users only see public-safe details of active accounts. Subject to the standard API rate limit.

Parameter	Location	Type	Description
id	URL	string	The target user's ID

Table 3.23: GET /api/user/:id — Parameters

Status Code	Condition	Description
200 OK	Success	Returns the user data (filtered based on requester role)
400 Bad Request	Missing parameter	User ID is required
401 Unauthorized	No/invalid token	Authentication failed
404 Not Found	User not found	No user exists with the given ID (or is disabled for non-admins)
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.24: GET /api/user/:id — Response Codes

Update My Profile

- **Endpoint:** PATCH /api/user/profile
- **Access:** USER, ADMIN
- **Description:** Updates basic profile details for the authenticated user. At least one field must be provided. The request is validated before processing. Subject to the standard API rate limit.

Field	Location	Type	Description
firstName	Body	string	Optional. New first name
lastName	Body	string	Optional. New last name
personalizations	Body	object	Optional. JSON object containing user personalizations

Table 3.25: PATCH /api/user/profile — Request Body

Status Code	Condition	Description
200 OK	Success	Returns the updated user profile
400 Bad Request	Empty body	No fields provided to update
401 Unauthorized	No/invalid token	Authentication failed
404 Not Found	Profile not found	User does not exist or is disabled
422 Unprocessable Entity	Validation failed	Invalid field values
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.26: PATCH /api/user/profile — Response Codes

Update My Email

- **Endpoint:** PATCH /api/user/profile/email
- **Access:** USER, ADMIN
- **Description:** Initiates an email change process. Updates the email, immediately revokes verification status, and sends a new OTP valid for 15 minutes. Also regenerates access and refresh tokens. Protected by a strict rate limiter to prevent email bombing (**3 requests per hour**).

Field	Location	Type	Description
email	Body	string	Required. The new email address

Table 3.27: PATCH /api/user/profile/email — Request Body

Status Code	Condition	Description
200 OK	Success	Email updated, code sent, and new tokens issued
400 Bad Request	Identical email	New email is identical to the current one
401 Unauthorized	No/invalid token	Authentication failed
404 Not Found	User not found	User does not exist or is disabled
409 Conflict	Email in use	The requested email is already registered
422 Unprocessable Entity	Validation failed	Invalid email format
429 Too Many Requests	Rate limit exceeded	Too many verification emails sent (Max 3 per hour)
500 Internal Server Error	Server error	Unexpected failure

Table 3.28: PATCH /api/user/profile/email — Response Codes

Update My Password

- **Endpoint:** PATCH /api/user/profile/password
- **Access:** USER, ADMIN
- **Description:** Updates the user's local password securely by verifying the old password first. Users authenticating via OAuth cannot use this route. Protected by a specific rate limiter to prevent brute force update attempts (**5 requests per hour**).

Field	Location	Type	Description
oldPassword	Body	string	Required. The user's current password
password	Body	string	Required. The new password
confirmPassword	Body	string	Required. Must match the new password

Table 3.29: PATCH /api/user/profile/password — Request Body

Status Code	Condition	Description
200 OK	Success	Password updated successfully
400 Bad Request	Invalid update	Passwords do not match, new is same as old, or OAuth user
401 Unauthorized	Authentication failed	Old password incorrect or token invalid
404 Not Found	User not found	User does not exist or is disabled
422 Unprocessable Entity	Validation failed	Password does not meet security requirements
429 Too Many Requests	Rate limit exceeded	Too many password update attempts (Max 5 per hour)
500 Internal Server Error	Server error	Unexpected failure

Table 3.30: PATCH /api/user/profile/password — Response Codes

Update User Role (Admin)

- **Endpoint:** PATCH /api/user/role/:id
- **Access:** ADMIN only
- **Description:** Updates a specific user's role. Admins are explicitly prevented from changing their own role to avoid accidental or malicious demotions. Subject to the standard API rate limit.

Field/Parameter	Location	Type	Description
id	URL	string	Required. The target user's ID
role	Body	string	Required. The new role to assign

Table 3.31: PATCH /api/user/role/:id — Parameters

Status Code	Condition	Description
200 OK	Success	Returns the user with the updated role
400 Bad Request	Missing role	Role field is required in the body
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Self-modification	Admins cannot change their own role, or non-admin user
404 Not Found	User not found	No user exists with the given ID
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.32: PATCH /api/user/role/:id — Response Codes

Delete My Profile

- **Endpoint:** DELETE /api/user/profile
- **Access:** USER, ADMIN
- **Description:** Self-deletion endpoint. Scrubs Personally Identifiable Information (PII), hard-deletes related relational assets (identity providers, credentials, nodes, projects), cleans up external assets (Cloudinary), and soft-deletes the user row. Subject to the standard API rate limit.

Status Code	Condition	Description
200 OK	Success	Profile and associated data deleted successfully
401 Unauthorized	No/invalid token	Authentication failed
404 Not Found	User not found	User does not exist or is already disabled
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.33: DELETE /api/user/profile — Response Codes

Delete User by ID (Admin)

- **Endpoint:** DELETE /api/user/:id
- **Access:** ADMIN only
- **Description:** Admin-facing deletion endpoint. Mirrors the self-deletion logic (PII scrubbing, cascading hard-deletes, Cloudinary cleanup, and soft-deletes the row) but acts on behalf of another user ID. Admins cannot delete their own account via this route. Subject to the standard API rate limit.

Parameter	Location	Type	Description
id	URL	string	The target user ID to delete

Table 3.34: DELETE /api/user/:id — Parameters

Status Code	Condition	Description
200 OK	Success	User and associated data deleted successfully
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Self-deletion	Cannot delete own admin account, or non-admin
404 Not Found	User not found	No user exists with the given ID
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.35: DELETE /api/user/:id — Response Codes

Resend Verification Code

- **Endpoint:** POST /api/user/resend-verification
- **Access:** USER, ADMIN
- **Description:** Resends a new OTP (Valid for 15 minutes) to an unverified user's email. Explicitly prevents disabled or deleted accounts from requesting emails. Protected by the email spam rate limiter (**3 requests per hour**).

Status Code	Condition	Description
200 OK	Success	New verification code sent via email
400 Bad Request	Already verified	User's email is already verified
401 Unauthorized	No/invalid token	Authentication failed
404 Not Found	User not found	User does not exist or is disabled
429 Too Many Requests	Rate limit exceeded	Too many verification emails sent (Max 3 per hour)
500 Internal Server Error	Server error	Unexpected failure

Table 3.36: POST /api/user/resend-verification — Response Codes

Confirm Email

- **Endpoint:** POST /api/user/confirm-email
- **Access:** USER, ADMIN
- **Description:** Verifies the email of the logged-in user using the OTP code sent to them. Upon successful verification, generates and returns new access and refresh tokens. Subject to the standard API rate limit.

Field	Location	Type	Description
code	Body	string/integer	Required. The numeric verification code

Table 3.37: POST /api/user/confirm-email — Request Body

Status Code	Condition	Description
200 OK	Success	Email confirmed successfully, returns new tokens
400 Bad Request	Invalid/Missing Code	Verification code is missing, non-numeric, or expired
401 Unauthorized	No/invalid token	Authentication failed
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.38: POST /api/user/confirm-email — Response Codes

Logout

- **Endpoint:** POST /api/user/logout
- **Access:** USER, ADMIN
- **Description:** Logs out the current user by clearing sensitive session data (refresh tokens, verification codes, and reset codes) from the database. Subject to the standard API rate limit.

Status Code	Condition	Description
200 OK	Success	User logged out successfully
401 Unauthorized	No/invalid token	Authentication failed
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.39: POST /api/user/logout — Response Codes

3.6.2.3 Project Endpoints

This section documents all API endpoints related to project management in FLOO. Projects are the core entity of the system — each project contains a workflow graph that defines the automation logic. All endpoints are protected by the **auth** middleware, requiring a valid JWT token, and the **restrictToVerified** middleware, ensuring that only users with verified email addresses can access these routes. Role-based access control is enforced via the **restrictTo** middleware. Additionally, all endpoints are subject to a global standard rate limit of 150 requests per 15 minutes. Exceeding this limit will result in a **429 Too Many Requests** response, while attempting to access these routes with an unverified account will yield a **403 Forbidden** response.

Get All Projects (Admin)

- **Endpoint:** GET `/api/project/all`
- **Access:** ADMIN only (Verified)
- **Description:** Returns all projects stored in the database with their associated user information. Supports filtering and pagination via query parameters.

Parameter	Location	Type	Description
<code>type</code>	Query	<code>string</code>	Filter projects by type
<code>active</code>	Query	<code>boolean</code>	Filter by active status (<code>true/false</code>)
<code>versionId</code>	Query	<code>integer</code>	Filter by version ID
<code>limit</code>	Query	<code>integer</code>	Number of results to return (default: 100)
<code>skip</code>	Query	<code>integer</code>	Number of results to skip for pagination (default: 0)

Table 3.40: GET `/api/project/all` — Query Parameters

Status Code	Condition	Description
200 OK	Success	Returns list of all projects with user info
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified / Non-admin	Insufficient permissions or email unverified
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.41: GET /api/project/all — Response Codes

Get My Projects

- **Endpoint:** GET /api/project/my-projects
- **Access:** USER, ADMIN (Verified)
- **Description:** Returns all projects belonging to the currently authenticated user. The user ID is extracted from the JWT token, so no ID parameter is required. Supports filtering and pagination.

Parameter	Location	Type	Description
type	Query	string	Filter projects by type
active	Query	boolean	Filter by active status (<code>true/false</code>)
versionId	Query	integer	Filter by version ID
limit	Query	integer	Number of results to return (default: 100)
skip	Query	integer	Number of results to skip for pagination (default: 0)

Table 3.42: GET /api/project/my-projects — Query Parameters

Status Code	Condition	Description
200 OK	Success	Returns list of the authenticated user's projects
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified user	User email is not verified
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.43: GET /api/project/my-projects — Response Codes

Get Projects by User ID (Admin)

- **Endpoint:** GET /api/project/all/user/:id
- **Access:** ADMIN only (Verified)
- **Description:** Returns all projects belonging to a specific user, identified by their ID in the URL. Returns a 404 error if the user does not exist. Supports filtering and pagination.

Parameter	Location	Type	Description
id	URL	string	The target user's ID
type	Query	string	Filter projects by type
active	Query	boolean	Filter by active status (true/false)
versionId	Query	integer	Filter by version ID
limit	Query	integer	Number of results to return (default: 100)
skip	Query	integer	Number of results to skip for pagination (default: 0)

Table 3.44: GET /api/project/all/user/:id — Parameters

Status Code	Condition	Description
200 OK	Success	Returns the user's projects
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified / Non-admin	Insufficient permissions or email unverified
404 Not Found	User not found	No user exists with the given ID
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.45: GET /api/project/all/user/:id — Response Codes

Get Project by ID

- **Endpoint:** GET /api/project/:id
- **Access:** USER, ADMIN (Verified, Strictly Owner)
- **Description:** Returns a single project by its ID. Access is strictly enforced requiring the authenticated user to be the owner of the project.

Parameter	Location	Type	Description
id	URL	string	The project ID

Table 3.46: GET /api/project/:id — Parameters

Status Code	Condition	Description
200 OK	Success	Returns the project data
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified user	User email is not verified
404 Not Found	Project not found / Unauthorized	No project exists with the given ID or the user is not the owner
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.47: GET /api/project/:id — Response Codes

Create Project

- **Endpoint:** POST /api/project
- **Access:** USER, ADMIN (Verified)
- **Description:** Creates a new project for the authenticated user. The request body is strictly validated using `createProjectValidators` ensuring the project name is provided (1-50 characters), the description does not exceed 200 characters, and the type is not empty. The user ID is taken from the JWT token automatically.

Field	Location	Type	Description
name	Body	string	Required. Project name (1-50 chars)
description	Body	string	Optional. Project description (max 200 chars)
type	Body	string	Required. Project type
versionId	Body	integer	Optional. Associated version ID (numeric)

Table 3.48: POST /api/project — Request Body

Status Code	Condition	Description
201 Created	Success	Project created, returns new project data
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified user	User email is not verified
422 Unprocessable Entity	Validation failed	Required fields missing, empty, or exceed character limits
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.49: POST /api/project — Response Codes

Delete Project

- **Endpoint:** DELETE /api/project/:id
- **Access:** USER, ADMIN (Verified, Strictly Owner)
- **Description:** Deletes a project by its ID. Access is strictly enforced so that only the project owner can delete it. Returns a 404 error if the project does not exist or does not belong to the requesting user.

Parameter	Location	Type	Description
id	URL	string	The project ID to delete

Table 3.50: DELETE /api/project/:id — Parameters

Status Code	Condition	Description
204 No Content	Success	Project deleted successfully
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified user	User email is not verified
404 Not Found	Project not found / Unauthorized	No project exists with the given ID or the user is not the owner
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.51: DELETE /api/project/:id — Response Codes

Update Project

- **Endpoint:** PATCH /api/project/:id
- **Access:** USER, ADMIN (Verified, Strictly Owner)
- **Description:** Updates the metadata of an existing project such as its name, description, type, or version. At least one field must be provided in the request body. Access is strictly limited to the project owner. The request is validated using `updateProjectValidators` enforcing length constraints (name up to 50 chars, description up to 200 chars, numeric version ID) before reaching the controller.

Field	Location	Type	Description
id	URL	string	Required. The project ID to update
name	Body	string	Optional. New project name (1-50 chars)
description	Body	string	Optional. New project description (max 200 chars)
type	Body	string	Optional. New project type
versionId	Body	integer	Optional. New version ID (numeric)

Table 3.52: PATCH /api/project/:id — Parameters

Status Code	Condition	Description
200 OK	Success	Returns updated project data
400 Bad Request	Empty body	No fields provided to update
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified user	User email is not verified
404 Not Found	Project not found / Unauthorized	No project exists with the given ID or the user is not the owner
422 Unprocessable Entity	Validation failed	Invalid field values, or character limits exceeded
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.53: PATCH /api/project/:id — Response Codes

Update Project Workflow

- **Endpoint:** PATCH /api/project/:id/workflow
- **Access:** USER, ADMIN (Verified, Strictly Owner)
- **Description:** Updates the workflow graph of a specific project. The workflow contains the full node and edge definitions of the automation. Access is strictly limited to the project owner. After saving the workflow, all node input images associated with the workflow's nodes are marked as non-draft (`isDraft = false`), committing them to the project. This endpoint is intentionally separate from the general update endpoint to isolate workflow logic from project metadata updates.

Field	Location	Type	Description
id	URL	string	Required. The project ID
workflow	Body	object	Required. The full workflow object (nodes & edges)

Table 3.54: PATCH /api/project/:id/workflow — Parameters

Status Code	Condition	Description
200 OK	Success	Returns updated project with new workflow
400 Bad Request	Missing workflow	No workflow object provided in body
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified user	User email is not verified
404 Not Found	Project not found / Unauthorized	No project exists with the given ID or the user is not the owner
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.55: PATCH /api/project/:id/workflow — Response Codes

3.6.2.4 Credential Endpoints

This section documents all API endpoints related to credentials management in FLOO, the visual AI automation tool. The endpoints are divided into Credential Types and User Credential Sets. All endpoints are protected by the **auth** and **restrictToVerified** middlewares, requiring a valid JWT token from a verified user. Role-based access control is enforced via the **restrictTo** middleware. Additionally, all endpoints are subject to a global standard rate limit of 150 requests per 15 minutes. Exceeding this limit will result in a 429 Too Many Requests response, while attempting to access these routes with an unverified account will yield a 403 Forbidden response.

Get All Credential Types

- **Endpoint:** GET /api/credential/type
- **Access:** USER, ADMIN (Verified)
- **Description:** Returns all supported credential types with their associated parameters, ordered by newest first.

Status Code	Condition	Description
200 OK	Success	Returns list of all credential types
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified user	User email is not verified
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.56: GET /api/credential/type — Response Codes

Get Credential Type by ID

- **Endpoint:** GET /api/credential/type/:id
- **Access:** USER, ADMIN (Verified)
- **Description:** Returns a single credential type and its parameters by its ID.

Parameter	Location	Type	Description
id	URL	string	The credential type ID

Table 3.57: GET /api/credential/type/:id — Parameters

Status Code	Condition	Description
200 OK	Success	Returns the credential type data
400 Bad Request	Missing ID	Type ID is required
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified user	User email is not verified
404 Not Found	Type not found	No credential type exists with the given ID
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.58: GET /api/credential/type/:id — Response Codes

Create Credential Type (Admin)

- **Endpoint:** POST /api/credential/type
- **Access:** ADMIN only (Verified)
- **Description:** Creates a new credential type and its associated parameters. The request body is strictly validated using `createCredentialTypeValidators`, ensuring the title is 1-100 characters long and the parameters array is not empty. It also validates each parameter to ensure titles are 1-100 characters, `maxChar` is a positive integer (if provided), and `isSensitive` is a boolean.

Field	Location	Type	Description
title	Body	string	Required. Credential type title (1-100 chars)
parameters	Body	array	Required. Non-empty array of parameter objects

Table 3.59: POST /api/credential/type — Request Body

Status Code	Condition	Description
201 Created	Success	Type created, returns new credential type data
400 Bad Request	Validation failed	Valid title and parameters array are required
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified / Non-admin	Insufficient permissions or email unverified
422 Unprocessable Entity	Validation failed	Validator checks failed (e.g., character limits exceeded, invalid types)
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.60: POST /api/credential/type — Response Codes

Delete Credential Type (Admin)

- **Endpoint:** DELETE /api/credential/type/:id
- **Access:** ADMIN only (Verified)
- **Description:** Deletes a credential type by its ID. Deleting a type cascades and deletes its associated parameters.

Parameter	Location	Type	Description
id	URL	string	The credential type ID to delete

Table 3.61: DELETE /api/credential/type/:id — Parameters

Status Code	Condition	Description
200 OK	Success	Type deleted successfully
400 Bad Request	Missing ID	Type ID is required
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified / Non-admin	Insufficient permissions or email unverified
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.62: DELETE /api/credential/type/:id — Response Codes

Update Credential Type (Admin)

- **Endpoint:** PATCH /api/credential/type/:id
- **Access:** ADMIN only (Verified)
- **Description:** Updates a credential type and its parameters. Operates inside a transaction to update the type and loop over parameters to update existing or create new ones.

Field	Location	Type	Description
id	URL	string	Required. The credential type ID
title	Body	string	Optional. New credential type title
oauth	Body	boolean	Optional. OAuth status
parameters	Body	array	Optional. Array of parameter objects to update/create

Table 3.63: PATCH /api/credential/type/:id — Parameters

Status Code	Condition	Description
200 OK	Success	Returns updated credential type data
400 Bad Request	Missing ID	Type ID is required
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified / Non-admin	Insufficient permissions or email unverified
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.64: PATCH /api/credential/type/:id — Response Codes

Get User Credential Sets

- **Endpoint:** GET /api/credential/set
- **Access:** USER, ADMIN (Verified)
- **Description:** Returns all credential sets belonging to the authenticated user. The user ID is automatically extracted from the JWT token. Sensitive values are decrypted and masked before returning to the frontend.

Parameter	Location	Type	Description
typeId	Query	string	Optional. Filter credential sets by type ID

Table 3.65: GET /api/credential/set — Query Parameters

Status Code	Condition	Description
200 OK	Success	Returns list of the authenticated user's credential sets
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified user	User email is not verified
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.66: GET /api/credential/set — Response Codes

Get User Credential Set by ID

- **Endpoint:** GET /api/credential/set/:id
- **Access:** USER, ADMIN (Verified, Strictly Owner)
- **Description:** Returns a specific user credential set by its ID. Access is strictly enforced to ensure the authenticated user owns the credential set. Sensitive parameter values are decrypted and masked.

Parameter	Location	Type	Description
id	URL	string	The credential set ID

Table 3.67: GET /api/credential/set/:id — Parameters

Status Code	Condition	Description
200 OK	Success	Returns the credential set data
400 Bad Request	Missing ID	Set ID is required
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified user	User email is not verified
404 Not Found	Set not found / Unauthorized	No set exists with the ID or user is not the owner
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.68: GET /api/credential/set/:id — Response Codes

Create User Credential Set

- **Endpoint:** POST /api/credential/set
- **Access:** USER, ADMIN (Verified)
- **Description:** Creates a new credential set for the authenticated user. Sensitive parameter values are automatically encrypted before being saved to the database. Validates that all provided parameters belong to the specified credential type.

Field	Location	Type	Description
typeId	Body	string	Required. The credential type ID
label	Body	string	Required. Credential set label
parameters	Body	array	Required. Array of parameter objects with values

Table 3.69: POST /api/credential/set — Request Body

Status Code	Condition	Description
201 Created	Success	Set created, returns new credential set data
400 Bad Request	Validation failed	Missing fields or parameters do not belong to type
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified user	User email is not verified
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.70: POST /api/credential/set — Response Codes

Update User Credential Set

- **Endpoint:** PATCH /api/credential/set/:id
- **Access:** USER, ADMIN (Verified, Strictly Owner)
- **Description:** Updates the label or parameters of an existing user credential set. Ensures parameters belong to the set and handles re-encryption of sensitive data. Access is strictly limited to the credential set owner.

Field	Location	Type	Description
id	URL	string	Required. The credential set ID to update
label	Body	string	Optional. New credential set label
parameters	Body	array	Optional. Array of parameter objects to update

Table 3.71: PATCH /api/credential/set/:id — Parameters

Status Code	Condition	Description
200 OK	Success	Returns updated credential set data
400 Bad Request	Validation failed	Missing ID or parameter doesn't exist for this type
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified user	User email is not verified
404 Not Found	Set not found / Unauthorized	No set exists with the ID or user is not the owner
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.72: PATCH /api/credential/set/:id — Response Codes

Delete User Credential Set

- **Endpoint:** DELETE /api/credential/set/:id
- **Access:** USER, ADMIN (Verified, Strictly Owner)
- **Description:** Deletes a user credential set by its ID. Access is strictly enforced so that only the owner can delete it. Deleting the set cascades and deletes all associated credential values.

Parameter	Location	Type	Description
id	URL	string	The credential set ID to delete

Table 3.73: DELETE /api/credential/set/:id — Parameters

Status Code	Condition	Description
204 No Content	Success	Credential set deleted successfully
400 Bad Request	Missing ID	Set ID is required
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified user	User email is not verified
404 Not Found	Set not found / Unauthorized	No set exists with the ID or user is not the owner
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.74: DELETE /api/credential/set/:id — Response Codes

3.6.2.5 Workflow Endpoints

This section documents all API endpoints related to workflow execution and management in FLOO. All workflow endpoints are protected by the `auth` middleware, requiring a valid JWT token, and the `restrictToVerified` middleware, ensuring that only verified users can execute or stop workflows. Role-based access control is enforced via the `restrictTo` middleware for `USER` and `ADMIN` roles. Additionally, these endpoints are subject to a specific execution rate limit of 30 requests per 1 minute to prevent spamming manual executions, on top of the global standard API rate limit of 150 requests per 15 minutes. Exceeding either limit will result in a `429 Too Many Requests` response, and attempting to access these routes with an unverified account will yield a `403 Forbidden` response.

Execute Workflow

- **Endpoint:** `POST /api/workflow/:id/execute`
- **Access:** `USER`, `ADMIN` (Verified)
- **Description:** Validates, compiles, and begins the execution of a project's workflow graph. The user ID is automatically extracted from the authenticated request.

Parameter	Location	Type	Description
<code>id</code>	URL	<code>string</code>	Required. The ID of the project workflow to execute

Table 3.75: `POST /api/workflow/:id/execute` — Parameters

Status Code	Condition	Description
200 OK	Success	Workflow started, returns updated project data
400 Bad Request	Missing ID	ProjectId is required
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified/Role	User is unverified or lacks permissions
429 Too Many Requests	Rate limit exceeded	Too many manual workflow executions
500 Internal Server Error	Server error	Unexpected failure

Table 3.76: POST /api/workflow/:id/execute — Response Codes

Stop Workflow

- **Endpoint:** POST /api/workflow/:id/stop
- **Access:** USER, ADMIN (Verified, Strictly Owner)
- **Description:** Halts a currently running workflow, stops all trigger nodes, cancels the active execution state, and marks the project as undeployed. Access is restricted to ensure the authenticated user owns the project.

Parameter	Location	Type	Description
id	URL	string	Required. The ID of the project workflow to stop

Table 3.77: POST /api/workflow/:id/stop — Parameters

Status Code	Condition	Description
200 OK	Success	Workflow stopped successfully, returns updated project
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified/Role	User is unverified or lacks permissions
404 Not Found	Project not found	Project with given ID not found for this user
409 Conflict	Invalid state	Project is inactive or not currently running
429 Too Many Requests	Rate limit exceeded	Too many manual workflow executions
500 Internal Server Error	Server error	Unexpected failure

Table 3.78: POST /api/workflow/:id/stop — Response Codes

3.6.2.6 Node Endpoints

This section documents all API endpoints related to node management and file operations in FLOO. All node endpoints are protected by the `auth` and `restrictToVerified` middlewares, requiring a valid JWT token from a verified user. Role-based access control is enforced via the `restrictTo` middleware for `USER` and `ADMIN` roles. Additionally, these endpoints are subject to two tiers of rate limiting: a global standard limit of 150 requests per 15 minutes for general operations, and a stricter limit of 20 requests per 15 minutes for file upload routes. Exceeding these limits will result in a `429 Too Many Requests` response, and attempting to access these routes with an unverified account will yield a `403 Forbidden` response.

Get All Nodes

- **Endpoint:** `GET /api/node/`
- **Access:** `USER`, `ADMIN` (Verified)
- **Description:** Returns all available nodes from the system registry.

Status Code	Condition	Description
200 OK	Success	Returns list of all node descriptions
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified user	User email is not verified
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.79: `GET /api/node/` — Response Codes

Get Nodes by Category

- **Endpoint:** `GET /api/node/category/:category`
- **Access:** `USER`, `ADMIN` (Verified)
- **Description:** Filters and returns nodes matching a specific category provided in the URL.

Parameter	Location	Type	Description
category	URL	string	Required. The category to filter nodes by

Table 3.80: GET /api/node/category/:category — Parameters

Status Code	Condition	Description
200 OK	Success	Returns nodes belonging to the requested category
400 Bad Request	Missing category	Category parameter is missing
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified user	User email is not verified
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.81: GET /api/node/category/:category — Response Codes

Get Trigger Nodes

- **Endpoint:** GET /api/node/triggers
- **Access:** USER, ADMIN (Verified)
- **Description:** Returns all nodes that act as triggers (`triggerNode === true`).

Status Code	Condition	Description
200 OK	Success	Returns list of all trigger nodes
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified user	User email is not verified
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.82: GET /api/node/triggers — Response Codes

Get Node Categories

- **Endpoint:** GET /api/node/categories
- **Access:** USER, ADMIN (Verified)
- **Description:** Returns the list of predefined node categories in the system.

Status Code	Condition	Description
200 OK	Success	Returns node categories
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified user	User email is not verified
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.83: GET /api/node/categories — Response Codes

Get Node by Name

- **Endpoint:** GET /api/node/:name
- **Access:** USER, ADMIN (Verified)
- **Description:** Returns a specific node by its exact name from the registry.

Parameter	Location	Type	Description
name	URL	string	Required. The exact name of the node

Table 3.84: GET /api/node/:name — Parameters

Status Code	Condition	Description
200 OK	Success	Returns the matching node data
400 Bad Request	Missing name	Name parameter is missing
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified user	User email is not verified
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.85: GET /api/node/:name — Response Codes

Upload Node Files

- **Endpoint:** POST /api/node/:nodeId/files
- **Access:** USER, ADMIN (Verified)
- **Description:** Uploads one or more files to Cloudinary and attaches them to a specific node as drafts. This route is subject to a strict upload limit of 20 requests per 15 minutes. The user must own the project associated with the upload.

Parameter/Field	Location	Type	Description
nodeId	URL	string	Required. The ID of the node
projectId	Body	string	Required. The ID of the associated project
files	FormData	file[]	Required. One or more files to upload

Table 3.86: POST /api/node/:nodeId/files — Parameters & Body

Status Code	Condition	Description
200 OK	Success	Files uploaded successfully
400 Bad Request	Missing inputs	Missing files, node ID, or project ID
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified / Owner	User unverified or does not own the project
404 Not Found	Project not found	The provided project ID does not exist
429 Too Many Requests	Upload limit exceeded	Too many file upload requests (Max 20/15min)
500 Internal Server Error	Server error	Unexpected failure

Table 3.87: POST /api/node/:nodeId/files — Response Codes

Get Node Files

- **Endpoint:** GET /api/node/:nodeId/files
- **Access:** USER, ADMIN (Verified)
- **Description:** Retrieves all files currently attached to a specific node (excluding drafts).

Parameter	Location	Type	Description
nodeId	URL	string	Required. The ID of the node
projectId	Query	string	Required. The associated project ID

Table 3.88: GET /api/node/:nodeId/files — Parameters

Status Code	Condition	Description
200 OK	Success	Returns list of active node files
400 Bad Request	Missing inputs	Missing node ID or project ID
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified user	User email is not verified
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.89: GET /api/node/:nodeId/files — Response Codes

Delete Node Files

- **Endpoint:** DELETE /api/node/:nodeId/files
- **Access:** USER, ADMIN (Verified)
- **Description:** Deletes all files associated with a specific node from both cloud storage and the database.

Parameter	Location	Type	Description
nodeId	URL	string	Required. The ID of the node
projectId	Query	string	Required. The associated project ID

Table 3.90: DELETE /api/node/:nodeId/files — Parameters

Status Code	Condition	Description
204 No Content	Success	Files deleted successfully, or no files found
400 Bad Request	Missing inputs	Missing node ID or project ID
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified user	User email is not verified
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.91: DELETE /api/node/:nodeId/files — Response Codes

Delete Single Node File

- **Endpoint:** DELETE /api/node/:nodeId/files/:fileId
- **Access:** USER, ADMIN (Verified)
- **Description:** Deletes a single, specific file belonging to a node from both cloud storage and the database.

Parameter	Location	Type	Description
nodeId	URL	string	Required. The ID of the node
fileId	URL	string	Required. The specific ID of the file to delete

Table 3.92: DELETE /api/node/:nodeId/files/:fileId — Parameters

Status Code	Condition	Description
204 No Content	Success	File deleted successfully
400 Bad Request	Missing inputs	Missing node ID or file ID
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified user	User email is not verified
404 Not Found	File not found	No file exists with the given ID
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.93: DELETE /api/node/:nodeId/files/:fileId — Response Codes

Get All Platform Files (Admin)

- **Endpoint:** GET /api/node/files/all
- **Access:** ADMIN only (Verified)
- **Description:** Returns all node files globally across the platform. Keeps track of system-wide file usage and ownership for auditing purposes. Supports pagination and filtering by draft status.

Parameter	Location	Type	Description
limit	Query	integer	Number of results to return (default: 50)
skip	Query	integer	Number of results to skip for pagination (default: 0)
isDraft	Query	boolean	Filter files by draft status (<code>true/false</code>)

Table 3.94: GET /api/node/files/all — Query Parameters

Status Code	Condition	Description
200 OK	Success	Returns platform files retrieved with ownership data
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified / Non-admin	Insufficient permissions or email unverified
429 Too Many Requests	Rate limit exceeded	Too many requests to the API
500 Internal Server Error	Server error	Unexpected failure

Table 3.95: GET /api/node/files/all — Response Codes

3.6.2.7 Execution Endpoints

This section documents all API endpoints related to workflow execution logs and analytics in FLOO. All execution endpoints are protected by the **auth** middleware, requiring a valid JWT token, and the **restrictToVerified** middleware, ensuring that only verified users can access execution logs or analytics. Role-based access control is enforced via the **restrictTo** middleware for **USER** and **ADMIN** roles. Additionally, these endpoints are subject to a specific polling rate limit of 120 requests per minute to prevent excessive load on the database, on top of the global standard API rate limit of 150 requests per 15 minutes.

Get Execution Analytics

- **Endpoint:** GET /api/execution/analytics
- **Access:** USER, ADMIN (Verified)
- **Description:** Returns aggregated execution metrics (total, success, failure rates) for the authenticated user, optionally filtered by project.

Parameter	Location	Type	Description
days	Query	integer	Optional. Number of days to look back (default: 7)
projectId	Query	string	Optional. Filter metrics by project ID

Table 3.96: GET /api/execution/analytics — Query Parameters

Status Code	Condition	Description
200 OK	Success	Returns execution metrics and status breakdown
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified user	User email is not verified
429 Too Many Requests	Rate limit exceeded	Too many requests (Max 120/min)
500 Internal Server Error	Server error	Unexpected failure

Table 3.97: GET /api/execution/analytics — Response Codes

Get All Project Executions

- **Endpoint:** GET /api/execution/projects/:projectId/executions
- **Access:** USER, ADMIN (Verified)
- **Description:** Retrieves a paginated list of all execution logs for a specific project.

Parameter	Location	Type	Description
projectId	URL	string	Required. The ID of the project
status	Query	string	Optional. Filter by status (e.g., SUCCESS, ERROR)
sortBy	Query	string	Optional. Sort field (createdAt/startedAt)
limit	Query	integer	Optional. Pagination limit (default: 50)
skip	Query	integer	Optional. Pagination skip (default: 0)

Table 3.98: GET /api/execution/projects/:projectId/executions — Parameters

Status Code	Condition	Description
200 OK	Success	Returns list of execution logs
400 Bad Request	Invalid params	Invalid status or missing ID
404 Not Found	Project not found	Project does not exist or access denied
429 Too Many Requests	Rate limit exceeded	Too many requests
500 Internal Server Error	Server error	Unexpected failure

Table 3.99: GET /api/execution/projects/:projectId/executions — Response Codes

Get Execution by ID

- **Endpoint:** GET /api/execution/projects/:projectId/executions/:executionId
- **Access:** USER, ADMIN (Verified)
- **Description:** Retrieves detailed information for a specific execution log.

Parameter	Location	Type	Description
projectId	URL	string	Required. The ID of the project
executionId	URL	string	Required. The ID of the execution

Table 3.100: GET /api/execution/projects/:projectId/executions/:executionId — Parameters

Status Code	Condition	Description
200 OK	Success	Returns execution details
404 Not Found	Not Found	Project or execution does not exist
429 Too Many Requests	Rate limit exceeded	Too many requests
500 Internal Server Error	Server error	Unexpected failure

Table 3.101: GET /api/execution/projects/:projectId/executions/:executionId — Response Codes

Delete Execution Log

- **Endpoint:** DELETE /api/execution/:executionId
- **Access:** USER, ADMIN (Verified)
- **Description:** Deletes a specific execution log. Only the project owner or an admin can perform this action.

Parameter	Location	Type	Description
executionId	URL	string	Required. The ID of the execution to delete

Table 3.102: DELETE /api/execution/:executionId — Parameters

Status Code	Condition	Description
204 No Content	Success	Execution log deleted successfully
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unauthorized	Insufficient permissions to delete log
404 Not Found	Not Found	Execution log not found
429 Too Many Requests	Rate limit exceeded	Too many requests

Table 3.103: DELETE /api/execution/:executionId — Response Codes

Get All Platform Executions (Admin)

- **Endpoint:** GET /api/execution/all
- **Access:** ADMIN only (Verified)
- **Description:** Retrieves a paginated list of all executions across the entire platform.

Parameter	Location	Type	Description
status	Query	string	Optional. Filter by execution status
limit	Query	integer	Optional. Page size (default: 50)
skip	Query	integer	Optional. Items to skip (default: 0)

Table 3.104: GET /api/execution/all — Query Parameters

Status Code	Condition	Description
200 OK	Success	Returns list of all platform executions
403 Forbidden	Non-admin user	Access denied
429 Too Many Requests	Rate limit exceeded	Too many requests

Table 3.105: GET /api/execution/all — Response Codes

3.6.2.8 Chatbot Endpoints

This section documents all API endpoints related to the FLOO AI chatbot. The chatbot allows users to interact with their project data via an LLM. All endpoints are protected

by the `auth` middleware, requiring a valid JWT token, and the `restrictToVerified` middleware, ensuring only verified users can access chat functionality. Role-based access control is enforced via the `restrictTo` middleware for `USER` and `ADMIN` roles.

Rate Limiting: To prevent LLM abuse and service spam, these endpoints are subject to an `aiChatLimiter` configured to 10 messages per minute. Exceeding this limit will result in a `429 Too Many Requests` response, and attempting to access these routes with an unverified account will yield a `403 Forbidden` response.

Get Chat Messages

- **Endpoint:** `GET /api/chat/:projectId/messages`
- **Access:** `USER`, `ADMIN` (Verified)
- **Description:** Retrieves the chat history for a specific project. The payload is limited to the most recent 100 messages to ensure performance.

Parameter	Location	Type	Description
<code>projectId</code>	URL	string	Required. The ID of the project

Table 3.106: `GET /api/chat/:projectId/messages` — Parameters

Status Code	Condition	Description
<code>200 OK</code>	Success	Returns the list of chat messages
<code>400 Bad Request</code>	Validation failed	<code>projectId</code> is missing
<code>401 Unauthorized</code>	No/invalid token	Authentication failed
<code>403 Forbidden</code>	Unverified user	User email is not verified
<code>404 Not Found</code>	Unauthorized/Missing	Project not found or user is not authorized
<code>429 Too Many Requests</code>	Rate limit exceeded	Too many requests sent

Table 3.107: `GET /api/chat/:projectId/messages` — Response Codes

Reply to Message

- **Endpoint:** `POST /api/chat/:projectId/messages`
- **Access:** `USER`, `ADMIN` (Verified)

- **Description:** Sends a new message to the chatbot, which is saved to the database. The system then fetches the last 50 messages to maintain LLM context window safety, generates an LLM reply, and saves the assistant’s response.

Field	Location	Type	Description
message	Body	string	Required. The user’s input text

Table 3.108: POST /api/chat/:projectId/messages — Request Body

Status Code	Condition	Description
200 OK	Success	Returns the LLM generated reply
400 Bad Request	Validation failed	Invalid project ID or message
401 Unauthorized	No/invalid token	Authentication failed
403 Forbidden	Unverified user	User email is not verified
404 Not Found	Unauthorized/Missing	Project not found or user is not authorized
429 Too Many Requests	Rate limit exceeded	Rate limit exceeded (10 msgs/min)

Table 3.109: POST /api/chat/:projectId/messages — Response Codes

3.6.2.9 Webhook Endpoints

This section documents the API endpoints related to external webhook integrations in FLOO. These endpoints allow external services to trigger workflow executions programmatically.

Unlike other API routes, these endpoints are public and do not require authentication, as they are intended to be called by external providers (e.g., GitHub, Typeform) that may not support JWT-based authorization. To prevent abuse, they are protected by a high-threshold rate limiter.

Run Workflow

- **Endpoint:** POST /api/webhook/:id
- **Access:** PUBLIC

- **Description:** Triggers the execution of a specific project workflow via a webhook. The system verifies that the project exists before enqueueing the execution. Subject to a loose rate limit of 1000 requests per minute to accommodate high-frequency event sources.

Parameter	Location	Type	Description
id	URL	string	Required. The ID of the project workflow to trigger

Table 3.110: POST /api/webhook/:id — Parameters

Status Code	Condition	Description
200 OK	Success	Workflow started successfully
400 Bad Request	Missing ID	workflowId is required
404 Not Found	Project missing	The project with the given ID does not exist
429 Too Many Requests	Rate limit exceeded	Too many requests, limit exceeded
500 Internal Server Error	Server error	Unexpected failure

Table 3.111: POST /api/webhook/:id — Response Codes

3.6.2.10 OAuth Endpoints

This section documents all API endpoints related to OAuth integrations in FLOO, the visual AI automation tool. These specific endpoints handle third-party service connections, primarily the Google Sheets API used for the Google Sheet node.

While the general API is subject to a standard limit of 150 requests per 15 minutes, the OAuth routes are protected by a stricter dedicated rate limit of 10 requests per 15 minutes to prevent authentication abuse. Exceeding this limit will result in a **429 Too Many Requests** response.

Initialize Google Sheets OAuth

- **Endpoint:** GET /api/oauth/google
- **Access:** USER, ADMIN

- **Description:** Initiates the OAuth 2.0 flow for Google Sheets. Protected by the `auth` middleware, requiring a valid JWT token. The endpoint extracts the `projectId` from the query, bundles it with the authenticated user's ID into a JSON stringified `state`, and redirects the user to the Google consent screen requesting offline access.

Parameter	Location	Type	Description
<code>projectId</code>	Query	<code>string</code>	Required. The ID of the project connecting to Google Sheets

Table 3.112: GET `/api/oauth/google` — Query Parameters

Status Code	Condition	Description
302 Found	Success	Redirects user to Google's OAuth consent screen
400 Bad Request	Missing parameter	Returns an error if <code>projectId</code> is missing
401 Unauthorized	No/invalid token	Authentication failed
429 Too Many Requests	Rate limit exceeded	Exceeded 10 requests per 15 minutes
500 Internal Server Error	Server error	Unexpected failure

Table 3.113: GET `/api/oauth/google` — Response Codes

Google Sheets OAuth Callback

- **Endpoint:** GET `/api/oauth/google/callback`
- **Access:** EXTERNAL (Google Redirect)
- **Description:** Handles the callback redirect from Google after the user grants consent. Authentication is handled via Passport's `google-sheets` strategy rather than standard JWT. If successful, the controller parses the returned state to retrieve the `projectId` and redirects the user back to the frontend workflow canvas.

Parameter	Location	Type	Description
state	Query/Body	string	Required. JSON string containing <code>userId</code> and <code>projectId</code>
code	Query	string	Required. The authorization code returned by Google

Table 3.114: GET `/api/oauth/google/callback` — Parameters

Status Code	Condition	Description
302 Found	Success	Redirects to <code>http://localhost:5173/workflow/:projectId</code>
400 Bad Request	Validation failed	Invalid state formatting, missing <code>projectId</code> , or missing credentials
401 Unauthorized	Auth failed	Passport strategy failed to authenticate the Google callback
429 Too Many Requests	Rate limit exceeded	Exceeded 10 requests per 15 minutes
500 Internal Server Error	Server error	Unexpected failure during callback processing

Table 3.115: GET `/api/oauth/google/callback` — Response Codes

3.6.3 Core Functional Flows

This section details the end-to-end execution flows for the primary features of the FLOO platform. By tracing the lifecycle of requests through our multi-layered architecture—from the initial API gateway to the database and external services—we define the standard operating procedure for our core features.

3.6.3.1 Authentication and Registration Flows

The authentication system is designed to provide secure, modular, and consistent user management. All authentication logic is centralized in the `auth.controller.ts`.

Register and Login Flow The registration and login processes share a common orchestration path. Validation occurs at the middleware layer, followed by controller

orchestration, which routes either to the service layer (on success) or the global error handler (on failure).

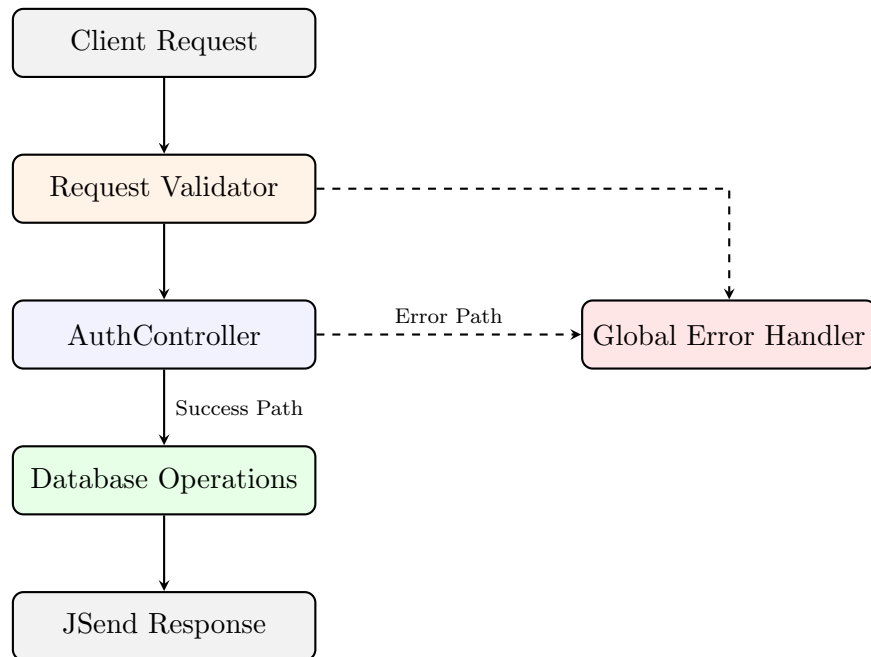


Figure 3.2: End-to-end flow for Registration and Login

- **Registration:** Upon receiving a request, the validator checks the payload. The `register` controller handles the hashing of passwords/OTPs and executes a transaction to store the user in Prisma. If an existing user is found or database operations fail, the flow routes to the `Global Error Handler`.
- **Login:** The `login` controller validates credentials against the database. Upon success, it returns tokens; on failure (e.g., incorrect credentials or disabled account), it throws an `AppError`, which is caught by the error handler.

3.6.3.2 Password Reset Flow

Password recovery utilizes a secure two-stage process to prevent unauthorized access.

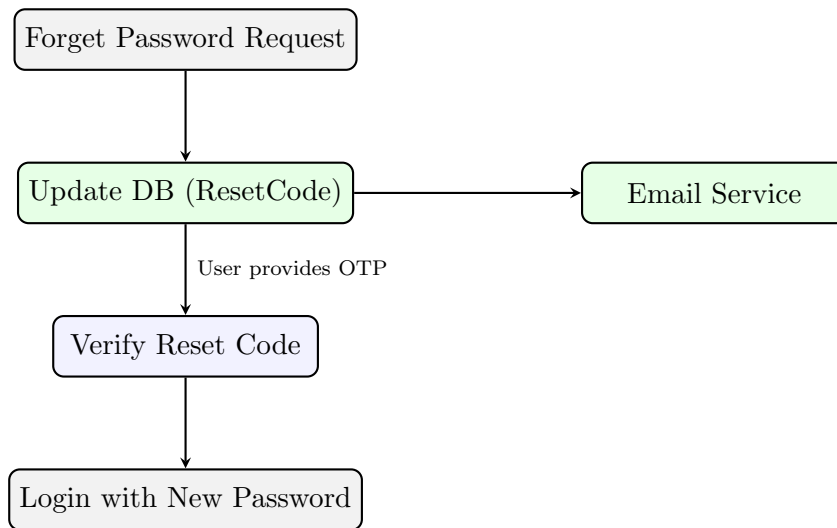


Figure 3.3: Password Reset lifecycle

1. **Initiation:** The `forgetPassword` controller generates a 15-minute OTP. To prevent user enumeration, it returns a generic success response regardless of the email's validity in the system.
2. **Verification:** The `verifyResetCode` controller hashes the new password and clears the reset code from the database upon success.

3.6.3.3 OAuth Authentication Flow

The OAuth implementation bridges external identity providers and our local application using a ticket-based exchange via Redis.

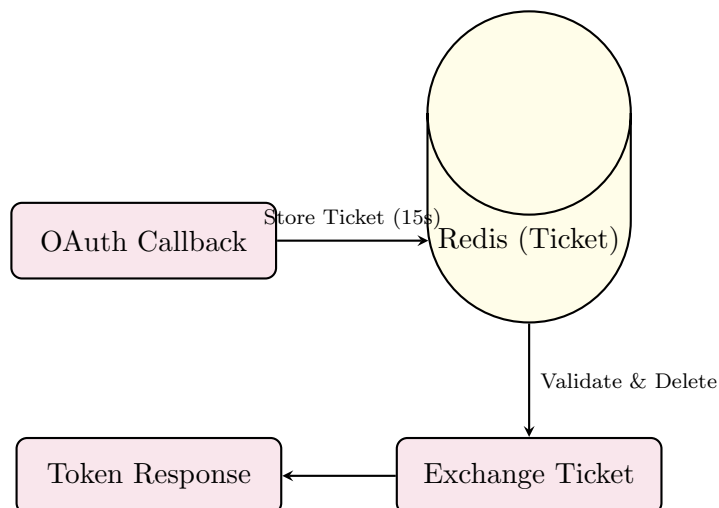


Figure 3.4: Ticket-based OAuth Exchange mechanism

- **Callback and Ticket Generation:** Upon successful provider authentication,

the callback controller creates a UUID-based "auth ticket" stored in Redis with a 15-second TTL.

- **Exchange:** The client (Web or Flutter) calls the `exchangeTicket` endpoint with the ticket. The backend validates the ticket against Redis, retrieves the user data, and immediately deletes the ticket to prevent re-use.

3.6.3.4 User Management Flows

The user management system focuses on secure profile maintenance and strict data hygiene during account deletion. All user-centric logic is centralized in `users.controller.ts`.

Profile Update Flow The profile update process relies on initial payload validation followed by atomic database updates.

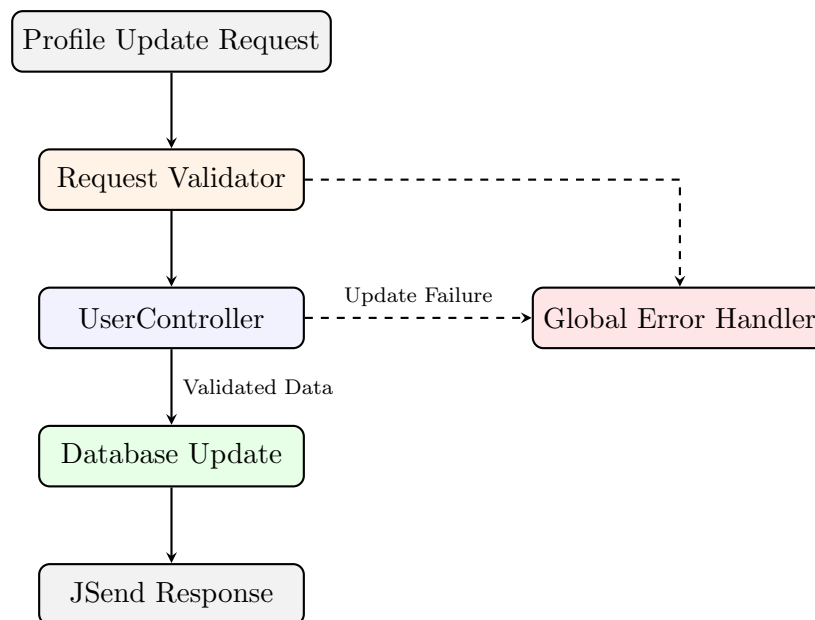


Figure 3.5: End-to-end flow for User Profile Updates

- **Orchestration:** Requests are routed through the controller, which verifies user existence and status.
- **Atomic Updates:** Sensitive operations are wrapped in `prisma.$transaction` to ensure that data modification and verification status revocation occur simultaneously.
- **Post-Update Logic:** For email changes, the flow triggers the email service and generates fresh security tokens.

Account Deletion Flow The account deletion process is designed to be comprehensive, ensuring the removal of PII and associated relational assets.

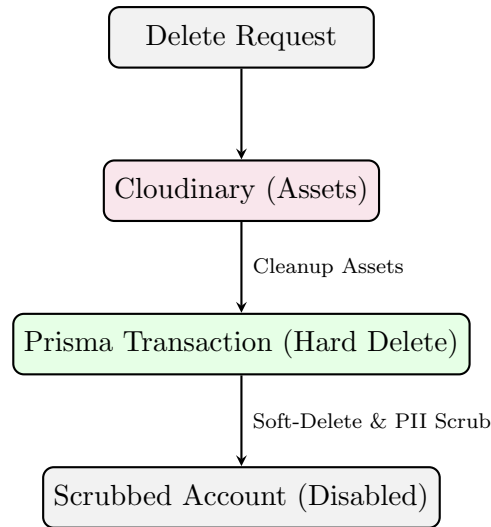


Figure 3.6: Account Deletion lifecycle

1. **Asset Cleanup:** The system invokes Cloudinary to remove user-specific files. This is decoupled from the database transaction.
2. **Cascading Hard-Delete:** The system executes multiple `deleteMany` calls on relational tables (Identity providers, credentials, node inputs, projects).
3. **PII Scrubbing:** The user record is disabled, and sensitive information is overwritten with dummy data to satisfy privacy requirements while maintaining referential integrity.

3.6.3.5 Project Management Flows

Project management in the FLOO platform relies on strict access control and centralized request sanitization. All project-related operations are handled by the `ProjectController`, which enforces data ownership and applies consistent filtering and pagination logic through a helper utility.

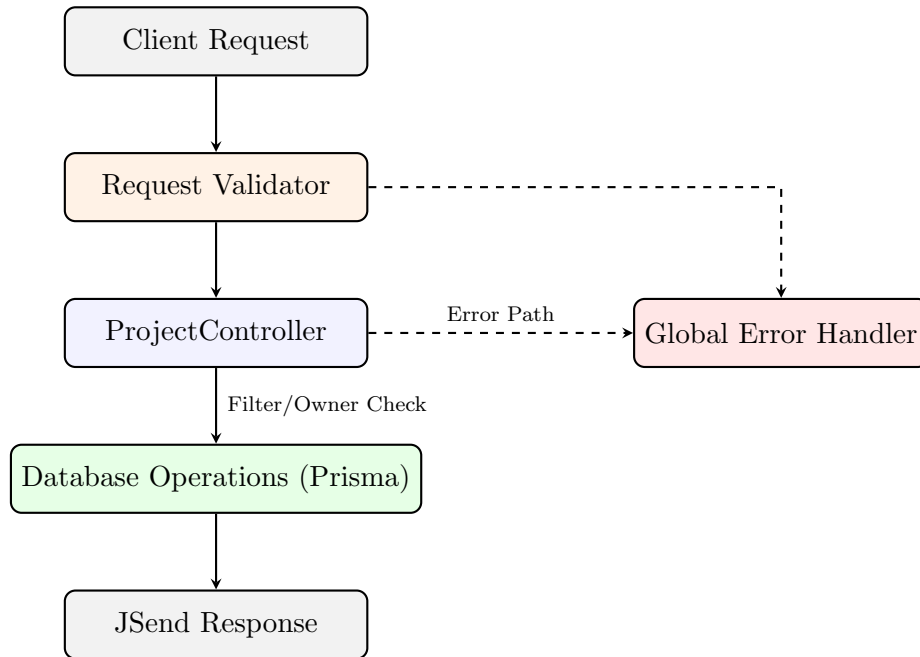


Figure 3.7: End-to-end flow for Project management operations

- **Filtering and Pagination:** All retrieval operations (`getProjectsByUserId`, `getAllProjects`, `getUserProjects`) utilize the `getProjectFilters` helper. This function parses query parameters into a typed `Prisma.ProjectWhereInput` object and standardizes `take` and `skip` values for pagination, ensuring consistency across all list endpoints.
- **Owner Enforcement:** Operations modifying or retrieving specific resources (`getProjectById`, `deleteProject`, `updateProject`, `updateWorkflow`) enforce strict ownership by including the authenticated `userId` in the Prisma `where` clause. This prevents unauthorized cross-user access at the database level.
- **Workflow Updates:** The `updateWorkflow` endpoint performs a multi-step update. After verifying project ownership, it updates the project workflow definition and executes a bulk update on the `NodeInput` table, marking all associated inputs as `isDraft: false` to finalize the deployment state.

3.6.3.6 Credential Management Flow

The credential management system is designed to securely handle sensitive user parameters, ensuring that secret data is encrypted before storage and masked upon retrieval. All credential logic is centralized in the `credentials.controller.ts`.

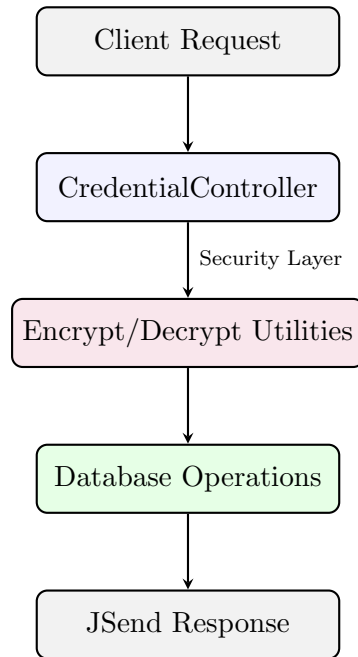


Figure 3.8: End-to-end flow for Credential Management

- **Credential Creation/Update:** Upon receiving a request, the controller coordinates with the `encrypt()` utility to secure sensitive parameters before performing Prisma transaction operations.
- **Credential Retrieval:** When fetching sets, the controller decrypts sensitive values using `decrypt()` and applies a `maskValue()` helper to ensure sensitive data is not exposed in the API response.
- **Data Integrity:** All operations are protected by authentication and verification middlewares to ensure only the owner of the credential set can access or modify it, maintaining strict ownership boundaries.

3.6.3.7 Workflow Execution Flow

The workflow execution flow manages the validation, graph construction, and job enqueueing process. It is orchestrated by the `WorkflowController`, which ensures the project exists and the user is authorized before invoking the core execution service.

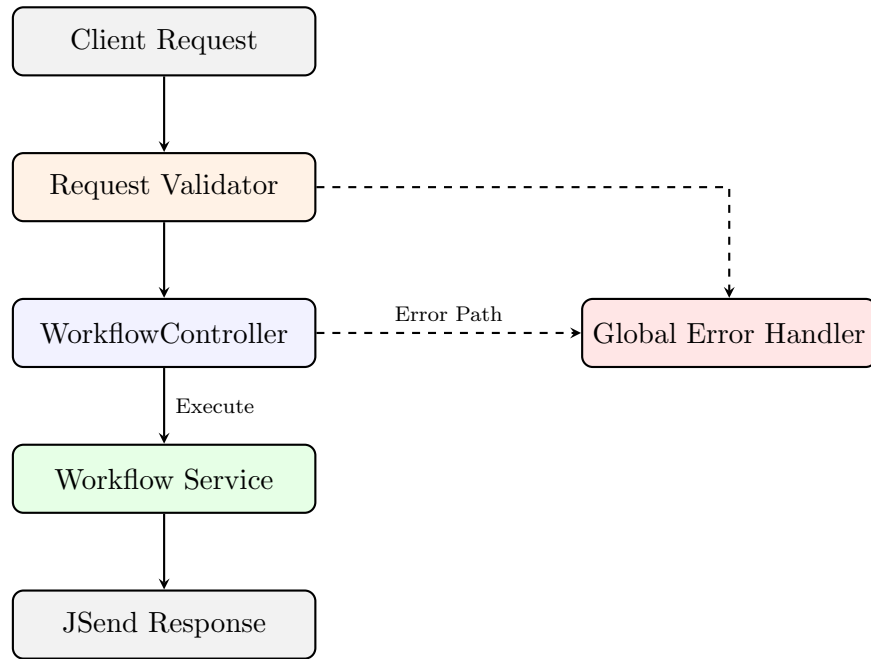


Figure 3.9: End-to-end flow for Workflow Execution

- **Validation:** Upon receiving the execution request, the controller verifies the presence of the `projectId` and performs authorization checks against the authenticated user.
- **Service Orchestration:** The controller calls `executeProjectWorkflow`, which handles the heavy lifting of graph compilation and task scheduling.
- **Error Handling:** Any failure, such as missing IDs or database connectivity issues during the execution setup, throws an `AppError`, which is caught and formatted by the `Global Error Handler`.

3.6.3.8 Node and File Management Flow

The node and file management flow handles both the retrieval of available system nodes from the registry and the end-to-end lifecycle of files associated with project nodes, including cloud storage uploads and database persistence.

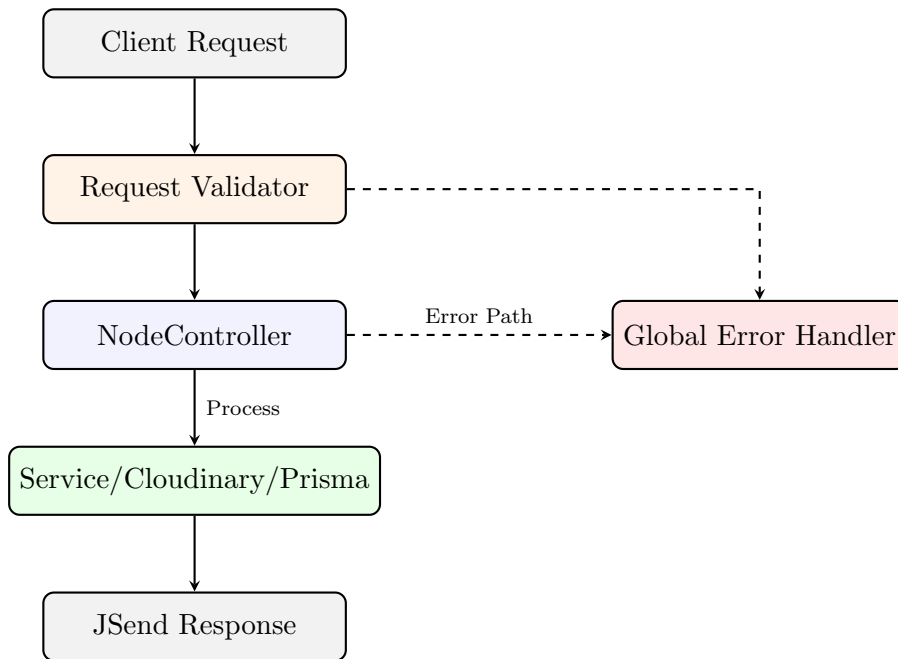


Figure 3.10: End-to-end flow for Node and File Operations

- **Registry Lookups:** For operations such as fetching nodes or categories, the controller directly interacts with the node registry service to provide the requested configuration data.
- **File Operations:** For file-related routes, the controller orchestrates the interaction between the database (Prisma) and cloud storage (Cloudinary). It enforces strict project ownership validation before initiating uploads or deletions.
- **Error Handling:** Any failure—such as file upload exceptions, unauthorized access to a project’s node, or missing parameters—is captured by the controller and delegated to the `Global Error Handler`.

3.6.3.9 Execution Management Flow

The execution management flow provides an interface for interacting with workflow logs, aggregated metrics, and administrative oversight. It ensures that users can access logs or delete executions only within the scope of projects they own, while providing administrative endpoints for platform-wide monitoring.

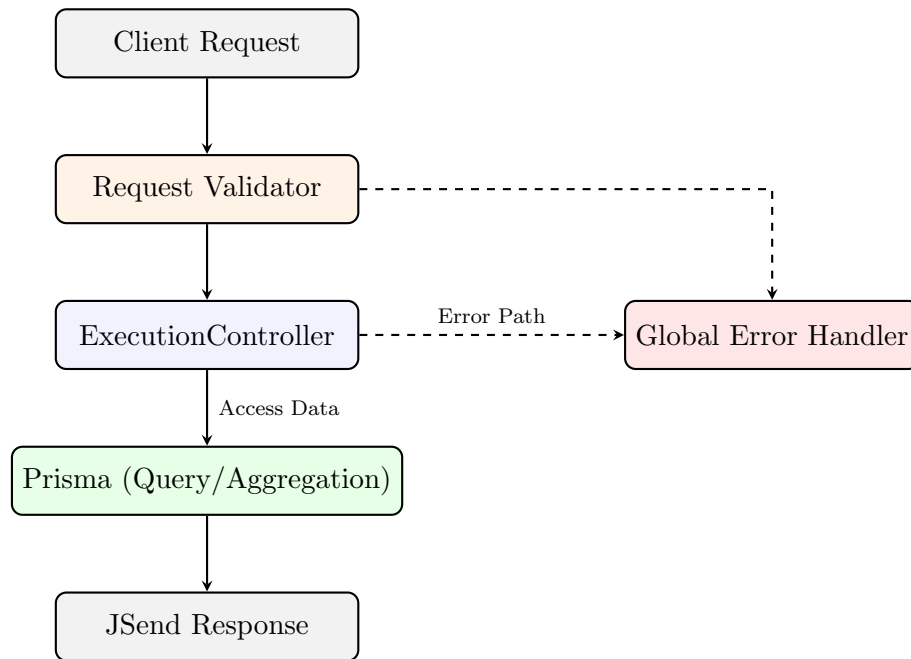


Figure 3.11: End-to-end flow for Execution and Metric Operations

- **Data Access and Filtering:** The controller processes incoming requests to retrieve specific executions or aggregated metrics, applying filters for project scope, status, and time range.
- **Security and RBAC:** Each request is validated against the authenticated user's ownership of the project. Administrative routes enforce a role-based access check (RBAC) to ensure only authorized users can delete logs or view platform-wide data.
- **Aggregation Logic:** The `getUserOverviewMetrics` endpoint leverages Prisma's `groupBy` functionality to compute success and failure rates dynamically within the database layer, ensuring efficient performance even across high volumes of execution logs.
- **Error Handling:** The controller utilizes centralized error handling for validation failures (e.g., invalid status, project not found) and security violations (e.g., unauthorized deletion attempts).

3.6.3.10 Webhook Execution Flow

The webhook execution flow enables external services to programmatically trigger project workflows. Unlike the standard API routes, these endpoints are public and do not require authentication, making them suitable for integrations with third-party platforms. They rely on high-threshold rate limiting to handle spikes in external event traffic while maintaining platform stability.

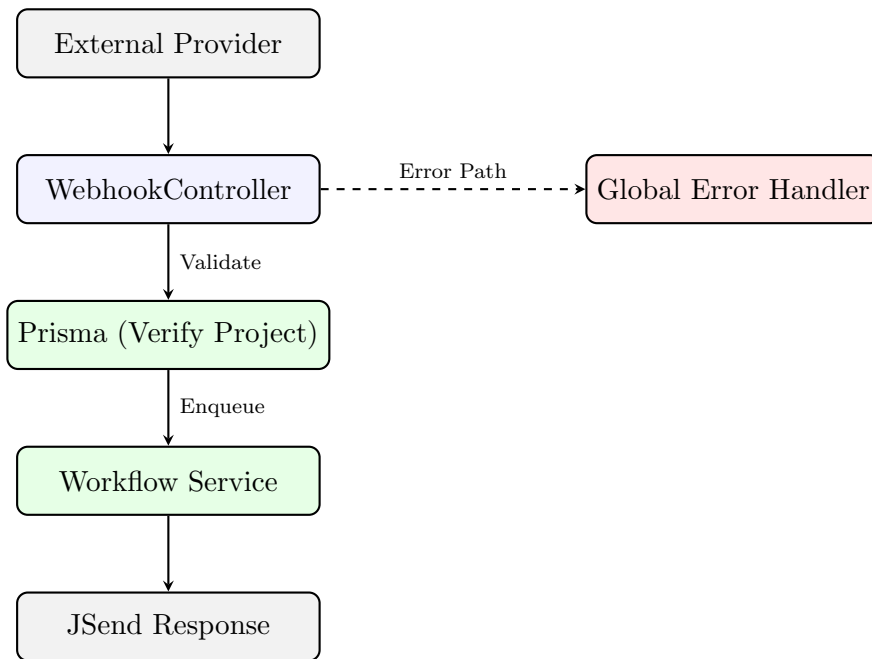


Figure 3.12: End-to-end flow for Webhook Workflow Triggers

- **Verification:** Upon receiving a request, the controller first verifies that the target project exists in the database using Prisma to prevent invalid execution triggers.
- **Execution Trigger:** Once the project is validated, the controller invokes the `executeProjectWorkflow` service to enqueue the job for processing.
- **Public Accessibility:** These endpoints are designed for high-frequency event sources (e.g., GitHub, Typeform). They bypass standard JWT requirements and are instead protected by specific high-threshold rate limiters to accommodate external spikes without blocking system usage.
- **Error Handling:** The controller handles workflow validation errors and missing project identifiers through the centralized error handling mechanism.

3.6.3.11 Google Sheets OAuth Controller Flow

The OAuth controller manages the Google Sheets API authentication callback lifecycle, specifically handling redirects from third-party identity providers. It is responsible for verifying credentials, parsing state information to maintain context (such as the `projectId`), and securely redirecting the user back to the appropriate frontend interface.

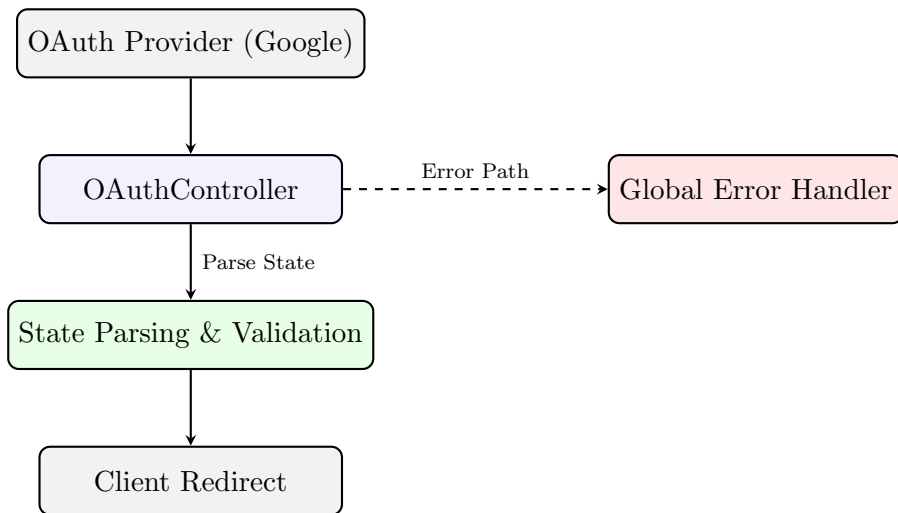


Figure 3.13: End-to-end flow for OAuth Callback handling

- **Credential Verification:** Upon receiving the callback, the controller ensures that the request contains valid user credentials provided by the OAuth strategy.
- **State Parsing:** The controller extracts the `projectId` from the `state` query parameter (or request body), enabling the application to resume the workflow context seamlessly.
- **Dynamic Redirection:** Once context is restored, the controller performs a 302 redirect to the frontend workflow canvas, ensuring the user is returned to the exact project they were interacting with.
- **Error Handling:** The controller handles invalid OAuth states or missing identifiers through the centralized error handling mechanism, ensuring failed authentication attempts are caught and reported clearly.

3.6.4 Middlewares

3.6.4.1 Authentication Middleware

The `auth` middleware authenticates requests by verifying the JWT provided in the Authorization header. If the token is missing, it falls back to a query parameter for OAuth redirects. The `restrictTo` middleware provides fine-grained Role-Based Access Control (RBAC).

```

1 export const auth = (req: Request, res: Response, next:
  NextFunction) => {
2   try {
3     let token = req.headers.authorization;
  
```

```

4      if (!token && req.query.token) token = req.query.token as
      string;
5      if (!token) throw new AppError("Unauthorized.", 401);
6
7      const payload = jwt.verify(token, process.env.
      ACCESS_TOKEN_PRIVATE_KEY!);
8      (req as AuthRequest).id = (payload as any).id;
9      (req as AuthRequest).role = (payload as any).role;
10     next();
11 } catch (error) { next(error); }
12 };
13
14 export const restricTo = (...roles: string[]) => {
15     return (req: Request, res: Response, next: NextFunction) => {
16         if (!roles.includes((req as AuthRequest).role))
17             return next(new AppError("Forbidden.", 403));
18         next();
19     };
20 };

```

3.6.4.2 Error Handler Middleware

This middleware acts as a facade for the error registry. It maps caught exceptions to their appropriate handlers based on the `canHandle` criteria.

```

1 export const errorHandler = (err: unknown, req: Request, res:
  Response, next: NextFunction) => {
2     const handler = errorHandlerRegistry.find(h => h.canHandle(err)
      );
3     if (handler) return handler.handle(err, res);
4 };

```

3.6.4.3 Verification Middleware

The `restrictToVerified` middleware checks the `isVerified` status on the request object, ensuring only accounts with confirmed emails can access protected features.

```

1 export const restrictToVerified = (req: Request, res: Response,
  next: NextFunction) => {
2     if (!(req as AuthRequest).isVerified) {
3         throw new AppError("Please verify your email to access this
      feature.", 403);

```

```
4   }
5   next();
6 };
```

3.6.4.4 JSend Middleware

The `jsend` middleware standardizes outgoing API responses by attaching helper methods (`success`, `fail`, `error`) to the Express Response object.

```
1 export function jsend(req: Request, res: Response, next:
  NextFunction) {
2   res.jsend = {
3     success: (msg, code = 200, data = null) =>
4       res.status(code).json({ status: "success", message: msg,
5         data }),
6     fail: (msg, code = 400, data = null) =>
7       res.status(code).json({ status: "fail", message: msg,
8         data}),
9     error: (msg, code = 500, data = null) =>
10      res.status(code).json({ status: "error", message: msg,
11        code, data }),
12   };
13   next();
14 }
```

3.6.4.5 Rate Limiter Middleware

This factory function generates IP-based rate limiters with configurable time windows and request maximums, throwing a 429 Too Many Requests error when limits are exceeded.

```
1 export const createRateLimiter = (minutes = 15, max = 100,
  message = "Too many requests.") => {
2   return rateLimit({
3     windowMs: minutes * 60 * 1000,
4     max,
5     handler: (req, res, next) => next(new AppError(message, 429))
6   });
7 };
```

3.6.4.6 Upload Middleware

The `uploadMiddleware` configures `multer` to handle multipart form data, specifically optimized for file uploads with a 5MB size limit per file.

```
1 const multerUpload = multer({
2   storage: multer.memoryStorage(),
3   limits: { fileSize: 5 * 1024 * 1024 },
4 });
5
6 export const uploadMiddleware = (req: Request, res: Response,
7   next: NextFunction) => {
8   multerUpload.array("files", 50)(req, res, (err) => {
9     if (err instanceof multer.MulterError) return next(new
10       AppError(err.message, 400));
11     else if (err) return next(err);
12     next();
13   });
14 }
```

3.6.4.7 Validation Middleware

The `validateRequest` middleware executes the results of the `express-validator` checks. If validation fails, it aggregates the errors and propagates them as a 422 Unprocessable Entity exception.

```
1 export const validateRequest = (req: Request, res: Response, next
2   : NextFunction) => {
3   const result = validationResult(req);
4   if (!result.isEmpty()) {
5     const errors = result.array().map(err => err.msg);
6     return next(new AppError('Validation errors', 422, errors));
7   }
8   next();
9 }
```

3.6.5 Validation Layer

Request validation is abstracted into its own layer using `express-validator`. Before a request payload is processed, it is checked against strict schemas to ensure data integrity and type safety. This “fail-fast” approach guarantees that the underlying controllers and services do not need to perform redundant sanity checks on incoming data .

3.6.5.1 Authentication Validators

These validators enforce security policies for user sessions and account recovery.

```
1 // auth.validators.ts
2 export const registerValidators = [
3   body('email').notEmpty().withMessage('Empty email').isEmail(),
4   body('password').isStrongPassword({ minLength: 8, ... }),
5   body('firstName').isLength({ min: 3, max: 15 }),
6   body('lastName').isLength({ min: 3, max: 15 })
7 ];
8
9 export const loginValidators = [
10  body('email').notEmpty().isEmail(),
11  body('password').notEmpty()
12 ];
13
14 export const verifyEmailValidators = [
15  body('email').notEmpty().isEmail(),
16  body('code').notEmpty().isNumeric()
17 ];
18
19 export const resetCodeValidators = [
20  body('email').notEmpty().isEmail(),
21  body('resetCode').notEmpty().isNumeric(),
22  body('password').notEmpty().isStrongPassword(...)
23 ];
```

Listing 3.1: Authentication validation schemas

3.6.5.2 Credential Validators

These ensure that credential definitions maintain proper parameter structures and validation constraints.

```
1 // credential.validator.ts
2 export const createCredentialTypeValidators = [
3   body('title').isLength({ min: 1, max: 100 }),
4   body('parameters').isArray({ min: 1 }),
5   body('parameters.*.title').isLength({ min: 1, max: 100 }),
6   body('parameters.*.maxChar').optional().isInt({ min: 1 }),
7   body('parameters.*.isSensitive').optional().isBoolean()
8 ];
```

Listing 3.2: Credential type validation schema

3.6.5.3 Project Validators

Project management routes utilize these validators to constrain metadata and versioning data.

```
1 // project.validators.ts
2 export const createProjectValidators = [
3   body('name').notEmpty().isLength({ min: 1, max: 50 }),
4   body('description').optional().isLength({ min: 1, max: 200 }),
5   body('type').notEmpty(),
6   body('versionId').optional().isNumeric()
7 ];
8
9 export const updateProjectValidators = [
10  body('name').optional().isLength({ min: 1, max: 50 }),
11  body('description').optional().isLength({ min: 1, max: 200 }),
12  body('type').optional().isLength({ min: 1 }),
13  body('versionId').optional().isNumeric()
14 ];
```

Listing 3.3: Project creation and update validation

3.6.5.4 User Validators

These validators secure user profile modifications, preventing invalid name formats, unauthorized email changes, or weak password updates.

```
1 // user.validators.ts
2 export const updateProfileValidators = [
3   body('firstName').optional().isLength({ min: 3, max: 15 }),
4   body('lastName').optional().isLength({ min: 3, max: 15 }),
5   body('personalizations').optional().isObject()
6 ];
7
8 export const updateEmailValidators = [
9   body('email').notEmpty().isEmail()
10 ];
11
12 export const updatePassword = [
```



```
13 body('password').notEmpty().isStrongPassword(...)
14 ];
```

Listing 3.4: User profile and account validation

3.6.6 Services

3.6.6.1 Email and Notification Services

Overview The email subsystem is designed to handle automated communications, such as user verification and password reset flows, in a decoupled and highly maintainable manner. It separates the concerns of email configuration, template management, and transport, allowing the application to add new email types or switch transport providers without modifying the core business logic.

Email Service Layer The email service layer relies on a modular architecture located in `backend/src/api/services/email/`, comprising four distinct components that manage the full lifecycle of an outgoing message.

Email Configuration The `email.config.ts` file serves as a centralized registry for all email types supported by the system. By mapping an `EmailType` to its specific subject and HTML template, the system ensures consistent messaging and simplifies the addition of new email types.

```
1 export const emailConfig = {
2   verify: {
3     subject: "Verify your email",
4     template: "verify-email",
5   },
6   reset: {
7     subject: "Reset your password",
8     template: "reset-password",
9   },
10 };
11
12 export type EmailType = keyof typeof emailConfig;
```

Listing 3.5: Email configuration registry (`email.config.ts`)

Template Loader The `templateLoader.ts` utility abstracts the filesystem operations required to handle HTML templates. It provides two core functions: `loadTemplate`,

which retrieves raw HTML from the `templates/` directory, and `renderTemplate`, which performs string interpolation to inject dynamic data (such as OTP codes or user names) into the templates before sending.

Transporter The `transporter.ts` file encapsulates the initialization of the Node-mailer transport mechanism. By isolating this logic, the system allows for the configuration of various SMTP drivers or API-based mail providers (e.g., SendGrid, Mailgun) in a single location, which the `email.service.ts` consumes to perform the actual network dispatch.

Email Service The `email.service.ts` acts as the primary orchestrator for the email subsystem. It exposes a single, clean function, `sendEmail`, which accepts the email type and dynamic data, looks up the configuration, renders the template, and invokes the transporter.

```
1 export const sendEmail = async (
2   type: EmailType,
3   to: string,
4   data: Record<string, string>
5 ) => {
6   // Get the email configuration for the specified type
7   const config = emailConfig[type];
8
9   const rawTemplate = loadTemplate(config.template);
10  const html = renderTemplate(rawTemplate, data);
11
12  // Send the email using the transporter
13  await transporter.sendMail({
14    to,
15    subject: config.subject,
16    html,
17  });
18 };
```

Listing 3.6: Email service orchestration (`email.service.ts`)

SMTP Service While the `email.service.ts` handles templated system emails, the `smtp.service.ts` (located in `backend/src/api/services/`) provides a lower-level, credential-based interface for sending arbitrary emails. This is critical for features where users must provide their own SMTP credentials (e.g., connecting a personal Gmail or Outlook account to an automation node).

The service handles the secure retrieval of encrypted SMTP credentials from the database, the construction of a dynamic Nodemailer transporter, and the execution of the mail send process.

```
1 export async function sendEmailSMTP(  
2   smtpData: SmtplibCredentialData,  
3   emailParams: SmtplibEmailParams,  
4 ) {  
5   try {  
6     const transporter = nodemailer.createTransport({  
7       host: smtpData.host,  
8       port: smtpData.port,  
9       secure: smtpData.secure,  
10      auth: {  
11        user: smtpData.username,  
12        pass: smtpData.password,  
13      },  
14    });  
15  
16    await transporter.sendMail({  
17      from: smtpData.username,  
18      to: emailParams.to,  
19      subject: emailParams.subject,  
20      text: emailParams.body,  
21    });  
22  
23    return { success: true };  
24  } catch (error: any) {  
25    throw new AppError('Failed to send email: ${error.message}',  
26      500);  
27  }  
}
```

Listing 3.7: SMTP execution flow (smtp.service.ts)

3.6.6.2 Cloud and AI Services

Cloudinary Service The Cloudinary service acts as the primary storage layer for the platform, handling file uploads and asset management. It leverages Node.js streams to pipe file buffers directly to Cloudinary's infrastructure, ensuring efficient memory usage.

Uploading Assets The `uploadToCloudinary` function encapsulates the logic for streaming file buffers. By using `cloudinary.uploader.upload_stream`, the system avoids loading large files entirely into RAM, which is critical for system stability.

```
1 export const uploadToCloudinary = async (folder: string,  
    fileBuffer: Buffer): Promise<UploadResult> => {  
2   return new Promise((resolve, reject) => {  
3     const stream = cloudinary.uploader.upload_stream(  
4       {  
5         folder,  
6         resource_type: "auto",  
7         use_filename: true,  
8         unique_filename: false,  
9         overwrite: true,  
10      },  
11      (error, result) => {  
12        if (error) {  
13          reject(new AppError('Upload failed: ${error.message}',  
14            500));  
15        } else {  
16          resolve({  
17            url: result!.secure_url,  
18            publicId: result!.public_id,  
19          });  
20        }  
21      });  
22      stream.end(fileBuffer);  
23    });  
24  };
```

Listing 3.8: Cloudinary upload logic (`cloudinary.service.ts`)

Resource Deletion Cloudinary requires an explicit `resource_type` for deletion (e.g., "image", "video", "raw"). The service includes a helper function, `toCloudinaryResourceType`, which derives this type from the asset's MIME type to ensure compatibility with Cloudinary's API requirements.

AI and LLM Services The AI infrastructure is bifurcated into a lightweight SDK wrapper (`groq.service.ts`) and a high-level service layer (`LLMChat.service.ts`) that orchestrates message formatting and prompt execution.

Groq Integration The `groq.service.ts` initializes the Groq SDK with the required API key from the environment. This serves as a singleton instance used throughout the backend for interacting with LLM models.

LLMChat Service The `LLMChat.service.ts` provides a simplified, polymorphic interface for the chat completion API. It supports function overloading, allowing developers to generate a reply either from a simple string or from a fully formed array of conversation messages.

```
1 // overload signatures
2 export async function generateReply(messages: Message[]): Promise
  <string>;
3 export async function generateReply(message: string): Promise<
  string>;
4
5 // implementation
6 export async function generateReply(input: Message[] | string):
  Promise<string> {
7   const messages: Message[] =
8     typeof input === "string"
9       ? [{ role: "user", content: input }]
10       : input;
11
12   const response = await groq.chat.completions.create({
13     model: "openai/gpt-oss-120b",
14     messages,
15   });
16
17   return response.choices[0]?.message?.content || "";
18 }
```

Listing 3.9: LLM Chat service with polymorphic input (`LLMChat.service.ts`)

This abstraction ensures that the business logic can consume AI responses without needing to handle the complexity of message formatting or model-specific configuration.

3.6.6.3 Execution and Node Management Services

Workflow Service The Workflow service is responsible for the orchestration and deployment lifecycle of project workflows. It acts as the bridge between the API controller and the execution engine.

Workflow Execution Lifecycle The `executeProjectWorkflow` function in `workflow.service.ts` handles the complete lifecycle of a workflow execution. It performs the following orchestration steps:

1. **Ownership & State Validation:** Verifies the project existence, authorization, and ensures the project is not already deployed.
2. **Graph Construction:** Invokes `getWorkflowGraph` to compile the raw JSON workflow payload into a structured `WorkflowGraph` instance.
3. **Pre-Execution Validation:** Validates the compiled graph to ensure there are no disconnected nodes or invalid sub-workflows using `validateAllWorkflows`.
4. **Deployment:** Updates the project status in the database and triggers the execution engine via `enqueueTriggerNodes`.

Node Registry Service The Node Registry service, defined in `nodes.service.ts`, provides a lightweight interface for the frontend to discover available automation nodes.

Registry Discovery The service leverages the `NodeRegistry` singleton to retrieve all available node classes. It processes these into a consumable format for the frontend UI:

```
1 export const getNodesFromRegistry = () => {
2   const registry = NodeRegistry.getInstance();
3   const nodes = registry.getAllnodes();
4   const nodeDescriptions: any[] = [];
5   for (const [key, NodeClass] of nodes) {
6     const description = (NodeClass as any).description;
7     nodeDescriptions.push({
8       ...description,
9       url: `https://res.cloudinary.com/dwtdta9o3/image/upload
10         /${description.name}.png`
11     });
12   }
13   return nodeDescriptions;
14 }
```

Listing 3.10: Node retrieval logic (`nodes.service.ts`)

This approach ensures that the frontend dynamically receives accurate node metadata, including display names, categories, and icon URLs, without hardcoding these values in the client-side application.

Node Output Service The Node Output service (`nodeoutput.service.ts`) manages the storage and persistence of files generated by nodes during execution.

File Handling and Persistence The service enforces strict validation rules—limiting file size to 5MB and a maximum of 50 files per request—before initiating the storage process.

- **Storage:** It iterates through the provided file buffer array and uses the `uploadToCloudinary` helper to store assets in a structured directory path: `users/{userId}/projects/{projectId}/`
- **Database Persistence:** Upon successful cloud upload, the service creates a record in the `NodeOutput` database table, linking the Cloudinary public ID and secure URL to the specific execution and node context.

This atomic orchestration ensures that every file tracked by the database is correctly backed by physical storage, maintaining data integrity for execution logs.

3.6.7 Authentication Strategies

3.6.7.1 Overview

The strategies layer encapsulates interchangeable authentication algorithms and service integrations using the Passport.js framework. This layer acts as the interface between the platform and external identity providers (IdPs) or service APIs, normalizing disparate authentication protocols into a unified internal workflow. By isolating these integrations, the platform can easily scale support for new services without modifying core business logic.

3.6.7.2 OAuth Identity Providers

The system implements three primary OAuth authentication strategies: Discord, GitHub, and Google. While each interacts with a different service provider, they adhere to a consistent internal contract defined by the following execution flow:

1. **Identity Verification:** Upon callback, the strategy queries the `AuthIdentityProvider` table in the database to determine if the provider-specific ID is already linked to an existing system user.
2. **User Upsert:** If no valid identity exists, the logic attempts to match the provider email against existing user records.
 - If a match is found, the strategy updates the existing account to link the new provider ID.

- If no match is found, a new user record is created with the default role of 'USER'.

3. **Audit Logging:** Every authentication attempt (successful or erroneous) is recorded in the `AuthHistory` table, which tracks the provider type, status, and error details for security monitoring.

```

1 // discord.strategy.ts
2 const findAuth = await prisma.authIdentityProvider.findUnique({
3   where: { provider: "discord", providerId: profile.id },
4   include: { user: true },
5 });
6
7 // Update or create user flow
8 if (!findAuth || findAuth.user.disabled) {
9   const existingUser = await prisma.user.findUnique({
10     where: { email },
11     include: { authIdentities: true }
12   });
13   // Link identity or create new user...
14 }

```

Listing 3.11: Common OAuth identity logic (Discord Strategy)

3.6.7.3 Google Sheets Integration

The Google Sheets strategy differs from the standard OAuth providers; it is a service-based strategy rather than an authentication-only strategy. Its primary responsibility is the secure management of third-party API credentials.

Credential Management This strategy handles the extraction of tokens during the OAuth callback and manages their persistence using the `userCredentialSet` and `userCredentialValue` models.

A critical requirement of this strategy is the security of stored tokens. The strategy explicitly invokes the encryption utility layer before persisting data to the database, ensuring that sensitive access and refresh tokens are never stored in plain text.

```

1 // googleSheet.strategy.ts
2 const encryptedAccessToken = encrypt(accessTokenValue);
3 await upsertCredentialValue(
4   credentialSet.id,
5   accessTokenParam.id,

```



```

6     encryptedAccessToken
7 );
8
9 if (refreshTokenValue && refreshTokenParam) {
10     const encryptedRefreshToken = encrypt(refreshTokenValue);
11     await upsertCredentialValue(
12         credentialSet.id,
13         refreshTokenParam.id,
14         encryptedRefreshToken
15     );
16 }

```

Listing 3.12: Credential encryption and storage flow

This architectural choice separates the concerns of account authentication from resource delegation, allowing the platform to maintain secure, persistent access to external resources like Google Sheets on behalf of the user.

3.6.8 Types

TypeScript interfaces and type definitions form the foundational contract of the application. This layer defines the exact shape of data structures used across the system, ensuring strict compile-time safety and catching structural mismatches between the frontend payloads, the internal API representation, and the execution engine.

3.6.8.1 Authentication Request (`authRequest.type.ts`)

This interface extends the standard Express `Request` object to inject custom user context. It is used by authentication middlewares to propagate identity information, including the user's unique identifier, role (used for RBAC), email, and their verification status throughout the request lifecycle.

```

1 import { Request } from 'express';
2
3 export interface AuthRequest extends Request {
4     id: string;
5     role: string;
6     email: string;
7     isVerified: boolean;
8 }

```

Listing 3.13: Authentication Request Interface

3.6.8.2 Google OAuth Profile (googleProfile.type.ts)

This interface standardizes the data structure returned by the Passport.js Google OAuth strategy. It maps the raw profile information into a typed object, allowing the application to securely extract user details like email, name, and locale during the OAuth callback process.

```
1 import { Profile } from "passport";
2
3 export interface GoogleProfile extends Profile {
4   _json: {
5     sub: string;
6     email: string;
7     email_verified: boolean;
8     name: string;
9     given_name: string;
10    family_name: string;
11    picture: string;
12    locale: string;
13  };
14 }
```

Listing 3.14: Google OAuth Profile Interface

3.6.9 Utils

The `utils` directory houses pure, stateless helper functions shared across multiple layers of the application. By centralizing these cryptographic and session-related operations, the platform prevents code duplication and ensures that critical security logic is handled consistently.

3.6.9.1 Hashing Utilities (hash.util.ts)

This utility abstracts password and data hashing using `bcryptjs`. It provides two asynchronous methods: `hash` to securely salt and hash strings (utilizing environment-configured salt rounds), and `compareHash` to verify plain-text inputs against existing hashes. This ensures that sensitive credentials are never stored in plaintext.

```
1 import bcrypt from 'bcryptjs'
2
3 const SALT_ROUNDS = Number(process.env.SALT_ROUNDS) || 10;
4
5 // Hash value
```

```

6 export const hash = async (value: string) => {
7   return await bcrypt.hash(value!, SALT_ROUNDS)
8 }
9
10 // Compare unhashed with hashed value
11 export const compareHash = async (value: string, hashed: string)
12   => {
13   return bcrypt.compare(value, hashed);
14 };

```

Listing 3.15: Hashing and Comparison Utility

3.6.9.2 JWT Token Management (jwtTokens.util.ts)

This utility manages the application's authentication lifecycle by signing JSON Web Tokens (JWT). It generates both a short-lived access token (2 hours) and a long-lived refresh token (14 days), both containing essential user metadata (ID, email, role, verification status) protected by environment-specific private keys.

```

1 import jwt from 'jsonwebtoken'
2
3 // Function to generate access and refresh tokens for login
4 export const generateTokens = (userId: string, userEmail: string,
5   userRole: string, isVerified: boolean) => {
6
7   const accessToken = jwt.sign(
8     { id: userId, email: userEmail, role: userRole, isVerified },
9     process.env.ACCESS_TOKEN_PRIVATE_KEY!,
10    { expiresIn: "2h" }
11  );
12
13   const refreshToken = jwt.sign(
14     { id: userId, email: userEmail, role: userRole, isVerified },
15     process.env.REFRESH_TOKEN_PRIVATE_KEY!,
16     { expiresIn: "14d" }
17   );
18
19   return { accessToken, refreshToken };
20 }

```

Listing 3.16: Token Generation Utility

3.6.9.3 OTP Generation (otp.util.ts)

This utility leverages `otplib` to handle one-time password (OTP) generation. It is primarily used during account registration, email updates, and password reset flows to generate time-sensitive tokens sent to the user's email, ensuring verification is authenticated and secure.

```
1 import { authenticator } from "otplib";
2
3 export const generateOtp = async () => {
4   const secret = authenticator.generateSecret();
5   return authenticator.generate(secret);
6 }
```

Listing 3.17: OTP Generation Utility

3.7 Error Handling Mechanism

3.7.1 Overview

The error handling mechanism provides consistent and maintainable error management across all layers of the application. It is built around a registry pattern that decouples error detection and response logic from the middleware layer, allowing new error types to be added without modifying any existing code.

3.7.2 Core Abstractions

3.7.2.1 The `IErrorHandler` Interface

The foundation of the system is the `IErrorHandler` interface, which defines a unified contract that all error handlers must fulfill:

- `canHandle(err: unknown): boolean` — determines whether this handler is responsible for the given error.
- `handle(err: unknown, res: Response): void` — sends the appropriate HTTP response for the error.

The use of `unknown` as the type for `err` in both methods ensures that every concrete handler fully honors the interface contract. Type narrowing happens internally inside each handler's `handle()` method, where the cast is safe because `canHandle()` has already confirmed the error type.

3.7.2.2 The `AppError` Class

`AppError` is a custom error class that extends the native JavaScript `Error`. It is used throughout the application to represent known, expected failures such as resource not found or unauthorized access. It carries three properties:

- `statusCode` — the HTTP status code for the response.
- `status` — derived automatically as `fail` for 4xx codes and `error` for 5xx codes.
- `payload` — optional additional data attached to the response.

3.7.3 The Handler Registry Pattern

3.7.3.1 Structure

Each error category is encapsulated in its own self-contained handler class. The system includes the following handlers:

- `PrismaKnownRequestErrorHandler` — handles known Prisma errors identified by error codes such as unique constraint violations (P2002), record not found (P2025), and connection errors (P1001, P1002).
- `PrismaValidationErrorHandler` — handles malformed queries sent to Prisma.
- `PrismaUnknownRequestErrorHandler` — handles unexpected Prisma request failures.
- `PrismaRustPanicErrorHandler` — handles critical Prisma engine panics.
- `PrismaInitializationErrorHandler` — handles database initialization failures, with specific internal logging per failure cause.
- `JwtErrorHandler` — handles JWT errors including invalid signatures, expired tokens, and tokens used before their valid time.
- `BcryptErrorHandler` — handles errors arising from invalid arguments passed to hashing and comparison operations.
- `NodemailerErrorHandler` — handles SMTP connection and authentication errors with specific internal logging per error code.
- `AppErrorHandler` — handles all intentionally thrown `AppError` instances from application code.
- `UnexpectedErrorHandler` — a catch-all fallback whose `canHandle()` always returns `true`. It must always be the last entry in the registry.

3.7.3.2 The Registry

The `errorHandlerRegistry` is an ordered array of handler instances. The middleware iterates through it and delegates to the first handler whose `canHandle()` returns `true`. Order is significant: specific handlers must precede general ones, and `UnexpectedErrorHandler` must always be last.

3.7.4 The Error Handler Middleware

The middleware contains no error-type-specific logic. It iterates the registry and delegates entirely:

```
1 export const errorHandler = (err, req, res, next) => {  
2   const handler = errorHandlerRegistry.find(h => h.canHandle(err)  
3   );  
4   if (handler) return handler.handle(err, res);  
5 }
```

3.7.5 Extending the System

Adding support for a new third-party library error requires two steps:

1. Implement the `IErrorHandler` interface in a new handler class, defining `canHandle()` for detection and `handle()` for the response.
2. Register the new handler in `errorHandlerRegistry` before `AppErrorHandler`.

For known, expected application failures such as not found or unauthorized, no new handler is needed. Throwing `new AppError(message, statusCode)` anywhere in the application is sufficient, as `AppErrorHandler` handles it automatically.

3.7.6 Design Evaluation

The error handling mechanism offers clear separation of concerns — each handler owns both the detection and response logic for exactly one error category. The middleware is fully decoupled from third-party libraries, depending only on the `IErrorHandler` abstraction. Each handler is independently testable, with unit tests verifying detection and response logic in isolation. Adding a new error type extends the system without modifying any existing code.

3.8 Workflow Graph Construction and Pre-Execution Validation

Before any workflow can be executed, the system must transform the raw JSON payload sent by the frontend into a structured, in-memory graph, and then verify that the graph is semantically correct. This process occurs in two successive phases: *graph construction* and *pre-execution validation*. Both phases complete before a single job is enqueued, ensuring that the execution engine only receives well-formed, executable workflows.

3.8.1 Data Model

The type system that underpins the entire workflow engine is defined in `workflow.type.ts`. Three core interfaces model the graph structure.

`INode` represents a single node in the workflow. Each node carries all the information required to execute it: its unique identifier, the node type name used to look it up in the registry, the project and execution identifiers, an `inputSource` policy that decides where the node reads its inputs from, the actual input and output data arrays, typed parameters, credential references, and the list of its resolved children.

```
1 export interface INode {
2   id:          string;    // unique node id within the workflow
3   type:        string;    // registry key, e.g. "cron", "poll"
4   label:       string;    // human-readable display name
5   projectId:   string;
6   executionId: string;    // assigned by the worker at runtime
7   inputSource: 'self' | 'parent' | 'combined' | 'any';
8   inputs:      string[];  // user-provided inputs for this node
9   parentOutputs: string[]; // outputs received from the upstream
                          node
10  parameters:  any[];
11  credentials?: { typeId: string; setId: string }[];
12  children:    INode[];   // resolved at deployment time
13 }
```

Listing 3.18: Core node interface (`workflow.type.ts`)

`IConnection` is deliberately minimal, capturing only the directed edge between two node identifiers:

```
1 export interface IConnection {
2   source: string;
3   target: string;
```

```

4 }
5
6 export interface IWorkflowGraph {
7     nodes:          INode[];
8     connections: IConnection[];
9 }

```

Listing 3.19: Connection and graph interfaces

`IWorkflowExecutionContext` is the object passed from the API controller to the workflow runner. It bundles the fully constructed graph with the project identifier so that downstream functions do not need to accept multiple unrelated arguments:

```

1 export interface IWorkflowExecutionContext {
2     workflowGraph: WorkflowGraph;
3     projectId:     string;
4 }

```

3.8.2 The WorkflowGraph Class

The `WorkflowGraph` class wraps an `IWorkflowGraph` value object with an *adjacency list* for efficient traversal, and exposes graph operations used by both the validation and execution layers.

3.8.2.1 Internal Representation

The class maintains two parallel data structures:

- `graph` — the plain `IWorkflowGraph` object, which stores the canonical lists of nodes and connections and is used wherever a serialisable snapshot of the graph is needed.
- `adjList` — a `Map<string, string[]>` that maps each node identifier to the identifiers of its direct successors. This is kept in sync with `graph.connections` whenever `addConnection` is called, allowing $O(1)$ neighbour lookup during traversal instead of scanning the full connection list each time.

```

1 export class WorkflowGraph {
2     graph:          IWorkflowGraph;
3     private adjList: Map<string, string[]>;
4
5     constructor() {
6         this.graph = { nodes: [], connections: [] };
7         this.adjList = new Map();
8     }
9 }

```



```

8   }
9
10  addNode(node: INode): void {
11      if (this.hasNode(node))
12          throw new AppError('Node with ID ${node.id} already exists
13              ', 400);
14      this.graph.nodes.push(node);
15  }
16
17  addConnection(edge: IConnection): void {
18      if (this.hasConnection(edge))
19          throw new AppError(
20              'Edge from ${edge.source} to ${edge.target} already
21                  exists', 400);
22
23      this.graph.connections.push(edge);
24
25      if (!this.adjList.has(edge.source))
26          this.adjList.set(edge.source, []);
27      this.adjList.get(edge.source)!.push(edge.target);
28  }
29  }

```

Listing 3.20: WorkflowGraph construction (`workflowGraph.class.ts`)

Both `addNode` and `addConnection` perform a duplicate check before mutating state, throwing an `AppError` if the node or edge is already present. This prevents malformed workflow payloads from producing silent inconsistencies in the graph.

3.8.2.2 Finding Trigger Nodes

A trigger node is structurally defined as a node with no incoming edges — it sits at the root of a connected component. `findTriggerNodes` identifies these nodes using an in-degree approach:

```

1  findTriggerNodes(): INode[] {
2      const allIDs      = this.getNodes().map(n => n.id);
3      const hasIncoming = new Set<string>();
4
5      // collect every node that appears as a target of some edge
6      for (const node of this.getNodes())
7          this.getNodeConnections(node.id).forEach(n => hasIncoming.add
            (n));

```

```

8
9 // nodes absent from hasIncoming have no parents -> they are
   triggers
10 const triggerIDs = allIDs.filter(id => !hasIncoming.has(id));
11 return this.graph.nodes.filter(n => triggerIDs.includes(n.id));
12 }

```

Listing 3.21: Trigger node detection by in-degree

The method builds the set of all nodes that appear as an edge target. Any node *not* in that set has an in-degree of zero and is therefore a candidate trigger. The subsequent validation step then confirms, via the node registry, that each such candidate is actually declared as a trigger node type.

3.8.2.3 Decomposing into Connected Components

A workflow canvas may contain multiple independent sub-workflows. Rather than validating and executing the entire graph as one unit, the system first decomposes it into *connected components* so that each sub-workflow can be validated and reported on individually. The decomposition uses BFS over the *undirected* version of the graph (edges are traversed in both directions):

```

1 findConnectedComponents(): WorkflowGraph[] {
2   const visited          = new Set<string>();
3   const components: WorkflowGraph[] = [];
4   const nodeMap          = new Map<string, WorkflowNode>(this.graph.nodes.map(n => [n.id, n]));
5   const reverseConnections = new Map<string, string[]>();
6
7   // build a reverse adjacency map to traverse edges backwards
8   for (const conn of this.graph.connections) {
9     if (!reverseConnections.has(conn.target))
10       reverseConnections.set(conn.target, []);
11     reverseConnections.get(conn.target)!.push(conn.source);
12   }
13
14   for (const startNode of this.graph.nodes) {
15     if (visited.has(startNode.id)) continue;
16
17     const componentGraph = new WorkflowGraph();
18     const queue          = [startNode];
19     visited.add(startNode.id);
20     componentGraph.addNode(startNode);

```

```

21
22     while (queue.length > 0) {
23         const current      = queue.shift()!;
24         const forward      = this.adjList.get(current.id) || [];
25         const backward     = reverseConnections.get(current.id) ||
26             [];
27         const neighbours   = [...new Set([...forward, ...backward])]
28             [];
29
30         for (const nId of neighbours) {
31             if (!visited.has(nId)) {
32                 visited.add(nId);
33                 componentGraph.addNode(nodeMap.get(nId)!);
34                 queue.push(nodeMap.get(nId)!);
35             }
36         }
37     }
38
39     // copy only the edges that belong to this component
40     for (const conn of this.graph.connections)
41         if (componentGraph.hasNode({ id: conn.source } as INode) &&
42             componentGraph.hasNode({ id: conn.target } as INode))
43             componentGraph.addConnection(conn);
44
45     components.push(componentGraph);
46 }
47
48 return components;
49 }

```

Listing 3.22: Connected component decomposition (BFS)

Treating the graph as undirected during decomposition is important. A directed BFS from a trigger node would miss any nodes upstream of a cycle or any disjoint nodes that happen to feed into the same component via reverse edges. Using both the forward adjacency list and the reverse map guarantees that every node is assigned to exactly one component regardless of edge direction.

3.8.3 Graph Construction from the HTTP Request

The `getWorkflowGraph` function in `workflow-pre-execution.validations.ts` is the entry point for turning a raw request payload into a typed `WorkflowGraph` instance. It is

called by the workflow controller immediately after the project record is fetched from the database.

3.8.3.1 Input Format

The frontend serialises the workflow canvas as a JSON object containing a **nodes** array and a **connections** array. Each node object follows the React Flow schema, where the node's semantic type is stored inside a nested **data.nodeName** field rather than at the top level:

```
1 {
2   "projectId": "abc-123",
3   "nodes": [
4     {
5       "id": "cron-1761747201552",
6       "data": { "nodeName": "cron", "label": "Scheduled Trigger"
7         },
8       "parameters": [
9         { "title": "interval", "value": 60000 },
10        { "title": "start", "value": 17000000000000 }
11      ],
12      "credentials": []
13    },
14    {
15      "id": "sheet-1761747208825",
16      "data": { "nodeName": "googleSheet", "label": "Append to
17        Sheet" },
18      "parameters": [...],
19      "credentials": [{ "typeId": "cb7c448b-...", "setId": "f3a1
20        ..." }]
21    }
22  ],
23  "connections": [
24    { "source": "cron-1761747201552",
25      "target": "sheet-1761747208825", "type": "custom-edge" }
26  ]
27 }
```

Listing 3.23: Example workflow payload from the frontend

3.8.3.2 Parsing and Validation

The function first validates the shape of the incoming data, then maps each raw node object to a typed `INode`, and finally populates the graph:

```
1 export const getWorkflowGraph =
2   (body: Record<string, unknown>): WorkflowGraph => {
3
4   // --- shape checks
5   -----
6   const nodesRaw = (body as any).nodes;
7   if (!Array.isArray(nodesRaw) ||
8     nodesRaw.some(n => typeof n.id !== 'string'))
9     throw new AppError('Invalid or missing nodes', 400);
10
11   const connectionsRaw = (body as any).connections;
12   if (!Array.isArray(connectionsRaw))
13     throw new AppError('Invalid or missing connections', 400);
14
15   const isValidConnections = connectionsRaw.every(conn =>
16     typeof conn.source === 'string' &&
17     typeof conn.target === 'string' &&
18     typeof conn.type === 'string'
19   );
20   if (!isValidConnections)
21     throw new AppError('Invalid connections', 400);
22
23   // --- mapping
24   -----
25   const nodes: INode[] = nodesRaw.map(n => ({
26     id:          n.id,
27     type:        n.data?.nodeName,      // unwrap from React Flow
28     schema:
29     label:       n.data?.label ?? '',
30     projectId:   body.projectId as string,
31     executionId: '',                    // assigned later by the
32     worker
33     inputSource: n.inputSource ?? 'parent',
34     inputs:      n.inputs,
35     parentOutputs: [],
36     parameters:  n.parameters ?? [],
37     credentials: n.credentials ?? []
```

```

34     children:      [],                                // resolved at deployment
        time
35   }));
36
37   const connections: IConnection[] = connectionsRaw.map(conn =>
    ({
38     source: conn.source,
39     target: conn.target,
40   }));
41
42   // --- graph population
    -----
43   const graph = new WorkflowGraph();
44   nodes.forEach(n => graph.addNode(n));
45   connections.forEach(c =>
46     graph.addConnection({ source: c.source, target: c.target }));
47
48   return graph;
49 };

```

Listing 3.24: Graph construction from request body

Two design details are worth noting. First, `executionId` is intentionally left empty at this stage. The worker assigns it later when it creates the `Execution` database record, and then propagates it through all child jobs. Second, `children` is left as an empty array; the workflow runner populates it recursively at deployment time via `buildChildren`, so that the full execution tree travels with each job without requiring further graph lookups.

3.8.4 Pre-Execution Validation Pipeline

Validation is orchestrated by `validateAllWorkflows`, which runs each connected component through a sequence of checks. The function returns a result object that separates valid sub-workflows from invalid ones, along with the specific error messages for each failure:

```

1 export const validateAllWorkflows =
2   async (workflowGraph: WorkflowGraph) => {
3
4   const validationResults = {
5     validWorkflows: [] as IWorkflowGraph[],
6     invalidWorkflows: [] as { workflow: IWorkflowGraph;
7                               errors: string[] }[],

```

```

8   };
9
10  const components = workflowGraph.findConnectedComponents();
11
12  for (const component of components) {
13      const errors: string[] = [];
14
15      const unregisteredNodes = checkAllNodesValidation(component);
16      const triggerResult      = validateTriggerNodes(component);
17
18      if (!triggerResult.ok)
19          errors.push(triggerResult.msg!);
20
21      if (unregisteredNodes.length > 0) {
22          errors.push('The following nodes are not registered: ' +
23              unregisteredNodes
24                  .map(n => `${n.name} (id: ${n.id})').join(', '));
25
26      } else if (triggerResult.ok && unregisteredNodes.length ===
27          0) {
28          // cycle check only runs when the graph structure is
29              already valid
30          const cycleResult = checkCycles(component);
31          if (!cycleResult.ok)
32              errors.push(cycleResult.msg!);
33      }
34
35      if (errors.length > 0)
36          validationResults.invalidWorkflows.push(
37              { workflow: component.graph, errors });
38      else
39          validationResults.validWorkflows.push(component.graph);
40  }
41
42  return validationResults;
43 };

```

Listing 3.25: Validation orchestrator

The three validators are applied in a deliberate order, described in the following subsections.

3.8.4.1 Trigger Node Validation

`validateTriggerNodes` enforces three structural rules:

1. The component must contain at least one node.
2. Every node with no incoming edges (as identified by `findTriggerNodes`) must be declared as a trigger node in the registry (`triggerNode: true`).
3. No node with incoming edges may be a trigger node — trigger nodes are only valid at the root of a component.

```
1 const validateTriggerNodes =
2   (graph: WorkflowGraph): { ok: boolean; msg?: string } => {
3
4   const triggerNodes = graph.findTriggerNodes();
5
6   if (graph.getNodes().length === 0)
7     return { ok: false, msg: 'No nodes found to execute.' };
8
9   if (triggerNodes.length === 0)
10    return { ok: false, msg: 'No start node found or cycle exists
11      .' };
12
13   // every root must be a registered trigger
14   for (const node of triggerNodes) {
15     const desc = registry.getDescription(node.type); // throws
16     if unregistered
17     if (!desc.triggerNode)
18       return { ok: false,
19         msg: 'Node `${node.type}` (id: `${node.id}`) is at the '
20         +
21         'start of the workflow but is not a trigger node.'
22       };
23   }
24
25   // no non-root node may be a trigger
26   for (const node of graph.getNodes()) {
27     if (triggerNodes.includes(node)) continue;
28     const desc = registry.getDescription(node.type);
29     if (desc.triggerNode)
30       return { ok: false,
```



```

27     msg: 'Trigger node `${node.type}` (id: `${node.id}`) ' +
28         'cannot be used in the middle of a workflow.' };
29 }
30
31 return { ok: true };
32 };

```

Listing 3.26: Trigger node structural validation

3.8.4.2 Node Registry Validation

`checkAllNodesValidation` ensures that every non-trigger node in the component is registered in the `NodeRegistry`. Trigger nodes are excluded from this check because their registration is already verified by `validateTriggerNodes`.

```

1  const checkAllNodesValidation =
2    (workflowGraph: WorkflowGraph): { name: string; id: string
3      }[] => {
4
5    const triggerNodeIds = new Set(
6      workflowGraph.findTriggerNodes().map(n => n.id));
7
8    const unregistered: { name: string; id: string }[] = [];
9
10   for (const node of workflowGraph.graph.nodes) {
11     if (triggerNodeIds.has(node.id)) continue; // skip triggers
12     if (!registry.has(node.type))
13       unregistered.push({ name: node.type, id: node.id });
14   }
15
16   return unregistered;
17 };

```

Listing 3.27: Registry membership check for non-trigger nodes

If any unregistered nodes are found, their names and identifiers are collected and returned as a list, which the orchestrator formats into a single error message. This allows the user to see all unregistered nodes in one response rather than having to fix them one by one.

3.8.4.3 Cycle Detection

The cycle check is the last to run and only executes if both preceding checks pass. This ordering is intentional: running a DFS-based cycle detector on a graph that contains

unregistered nodes or a missing trigger would produce unreliable or misleading results, since the graph structure cannot be trusted.

Cycle detection uses a standard three-colour DFS algorithm, encoded by the `VisitState` enum:

```
1 export enum VisitState {
2   UNVISITED = 0, // not yet reached
3   VISITING  = 1, // currently on the DFS call stack
4   VISITED   = 2, // fully processed, no cycle through this node
5 }
```

Listing 3.28: `VisitState` enum (`workflow.enum.ts`)

```
1 const checkCycles =
2   (graph: WorkflowGraph): { ok: boolean; msg?: string } => {
3
4   const visited = new Map<string, VisitState>(
5     graph.getNodes().map(n => [n.id, VisitState.UNVISITED]));
6
7   const dfs = (u: string): boolean => {
8     visited.set(u, VisitState.VISITING);
9
10    for (const v of graph.getNodeConnections(u)) {
11      if (visited.get(v) === VisitState.VISITING) return true;
12      // back edge
13      if (visited.get(v) === VisitState.UNVISITED && dfs(v))
14        return true;
15    }
16
17    visited.set(u, VisitState.VISITED);
18    return false;
19  };
20
21  for (const node of graph.getNodes())
22    if (visited.get(node.id) === VisitState.UNVISITED)
23      if (dfs(node.id))
24        return { ok: false, msg: 'Workflow has cycles' };
25  };
26  return { ok: true };
```

Listing 3.29: DFS cycle detection

A node in the **VISITING** state is one that is currently on the recursive call stack. Encountering such a node while traversing its own descendants means a back edge has been found, which is the defining characteristic of a cycle in a directed graph. Once all descendants of a node have been processed without finding a back edge, the node is marked **VISITED** and will not be re-explored. The outer loop ensures that all disconnected components — if `findConnectedComponents` was bypassed — are also checked.

3.8.5 Validation Ordering and Error Accumulation

The order in which the three validators run is not arbitrary. Table ?? summarises the rationale for their sequence.

Order	Validator	Rationale
1	<code>validateTriggerNodes</code>	Structural check. A component without a valid trigger root cannot be executed at all, and a trigger node in the middle of a graph is a logical error that makes the remaining checks meaningless.
2	<code>checkAllNodesValidation</code>	Registry check. An unregistered node would cause the worker to throw at run-time. Catching this statically produces a clearer error and avoids wasting a job queue slot.
3	<code>checkCycles</code>	Graph-theoretic check. Only runs when the graph is structurally sound. A cyclic graph would cause <code>buildChildren</code> to recurse infinitely and flood the job queue, so cycles must be rejected before deployment.

Table 3.116: Pre-execution validation order and rationale

Errors from the trigger and registry checks are accumulated independently and both reported if both fail. The cycle check is *gated* behind the registry check: it only executes if all non-trigger nodes are registered, since a DFS over a graph that references unknown node types cannot be trusted to reflect the true structure of the intended workflow.

Once the full validation result is returned to the workflow controller, any component listed in `invalidWorkflows` causes the entire deployment request to be rejected with a structured error payload identifying the problematic nodes and the reasons for failure.

Margin
note.

Appendix A

Extra Information

Some more text ...

